Rapport de stage de fin d'études

Master Systèmes et Applications Réparties

Sorbonne Université

Conception et développement d'un tableau de bord opérationnel KPI distribué et scalable

Soufiane El Mouahid

Entreprise d'accueil : Mubea, Wendlingen am Neckar (Allemagne)
Maître de stage : Dr. Stefan Kompa
Tuteur académique : Alessio Pagliari
Période du stage : Avril 2025 – Août 2025

Année universitaire 2024–2025

Remerciements

Je tiens à remercier chaleureusement l'équipe de Mubea Wendlingen pour son accueil remarquable. Des personnes disponibles, bienveillantes et professionnelles, toujours prêtes à répondre à mes questions, à l'écoute et respectueuses. Je me suis très vite senti pleinement intégré au sein de l'équipe.

Je remercie tout particulièrement le Dr Stefan Kompa, mon superviseur, pour son accompagnement constant tout au long du stage. Il a pris en charge chacune de mes demandes, a veillé à ma progression et à mon bien-être, et a toujours agi dans mon intérêt. Il m'a accordé une réelle confiance et une considération sincère, tant pour ma personne que pour mon travail. Très rapidement, je me suis senti davantage comme un collègue — et même un ami — que comme un simple stagiaire.

Table des matières

1 Introduction **1**

2 Entreprise et contexte du stage **1**
2.1 Présentation de l'entreprise . . . 1
2.2 Contexte du stage . . . 2

3 Problématique et notions de base **2**
3.1 Problématique industrielle . . . 3
3.2 Définitions . . . 3
3.2.1 KPI . . . 3
3.2.2 Operational KPI Dashboard . . . 4
3.2.3 Pipeline . . . 4
3.2.4 Couches MES / ERP / PLC . . . 4

4 État de l'art **5**

5 Conception et modélisation de la solution **6**
5.1 Technologies utilisées . . . 6
5.1.1 Python . . . 6
5.1.2 C++ . . . 7
5.1.3 Redis . . . 7
5.1.4 InfluxDB . . . 7
5.1.5 Grafana . . . 8
5.2 Architecture globale . . . 8
5.3 Distributivité et scalabilité . . . 9
5.3.1 Distributivité . . . 9
5.3.2 Scalabilité . . . 9
5.4 Schéma d'architecture . . . 10

6 Développement et modules **10**
6.1 Architecture et hiérarchie du code . . . 11
6.2 Fonctionnement global et multi-threading des modules . . . 11
6.3 Synchronisation (`sharedcontrol.hpp`) . . . 12
6.4 Indépendance des modules . . . 14
6.4.1 Main et fichier de configuration . . . 14
6.4.2 Arrêt centralisé . . . 14
6.5 Connector . . . 15
6.6 Convertor . . . 16
6.7 PreRouter . . . 17
6.8 Router . . . 17
6.9 Kpi_engine . . . 19
6.10 Kpi_pusher . . . 20
6.11 Stockage InfluxDB . . . 20
6.12 Front-end Grafana . . . 21

7 Validation et benchmarks **22**
7.1 `main.cpp` (lanceur du prototype) . . . 22
7.2 `json_generator.py` . . . 23
7.3 Tests et mesures de performances . . . 24
7.3.1 Environnement de test . . . 24
7.3.2 Tests et validation fonctionnelle . . . 25

7.3.3 Benchmarks (débit end-to-end) 26

8 Conclusion **27**

9 Perspectives et améliorations possibles **27**

9.1 Orchestrateur 27
9.2 Optimisations techniques 28
9.3 Sécurité et robustesse 29
9.4 Automatisation 29
9.5 Documentation technique 30

Annexes **30**

Références **36**

1 Introduction

Dans un contexte industriel où l'optimisation des procédés de production et la réactivité face aux aléas sont devenues essentielles, la mesure et l'exploitation des indicateurs de performance (Key Performance Indicators – KPI) jouent un rôle central. Les entreprises multinationales comme Mubea, fournisseur dans le secteur automobile, doivent non seulement assurer une qualité constante et une productivité élevée, mais aussi disposer d'outils de suivi efficaces et adaptables à des environnements de production variés.

Dans ce contexte, la question directrice de mon travail est la suivante : *comment monitorer l'ensemble des chaînes de production de Mubea à l'échelle mondiale au travers de KPI, représentés par des visualisations explicites, en temps réel, et de manière suffisamment flexible pour intégrer de nouveaux indicateurs à l'avenir.*

Pour y répondre, mon stage de fin d'études au sein du site de Wendlingen am Neckar avait pour objectif principal de concevoir et développer un tableau de bord opérationnel de KPI adaptable, distribué et scalable, capable d'être déployé à terme sur l'ensemble des sites de production du groupe. Ce projet nécessitait, en premier lieu, une compréhension approfondie des processus industriels et des besoins métiers liés au suivi de la production. Il s'agissait ensuite d'établir un lien entre ces besoins et des indicateurs pertinents, puis de mettre en place une architecture logicielle robuste et évolutive permettant de les calculer, de les stocker et de les visualiser en temps réel.

En complément de cet objectif, j'ai apporté un soutien technique ponctuel aux équipes locales, en proposant et en développant des solutions informatiques adaptées aux besoins immédiats du site. Ces missions m'ont amené à collaborer avec divers membres de l'équipe, à comprendre leurs contraintes, à élaborer des outils répondant à leurs problématiques, puis à documenter ces solutions afin d'en assurer la pérennité.

Le présent rapport présente d'abord l'entreprise d'accueil et le contexte du stage, puis expose les notions de base et la problématique, en situant la démarche dans l'état de l'art. Il décrit ensuite les choix technologiques, la conception et l'architecture de la solution, détaille les modules et leur développement, et rend compte de la validation et des *benchmarks*. Il se termine par une conclusion, puis ouvre sur les perspectives d'amélioration.

2 Entreprise et contexte du stage

Cette section présente d'abord Mubea et son positionnement dans l'automobile, puis précise le cadre opérationnel du stage : site concerné, spécialisation, organisation de la production et environnement SI (*Système d'Information*).

2.1 Présentation de l'entreprise

Mubea (Muhr und Bender KG) est un équipementier automobile allemand de rang 1, entreprise familiale fondée en 1916 et basée à Attendorn. Acteur global de la chaîne d'approvisionnement automobile, Mubea conçoit et industrialise des composants techniques pour les véhicules et collabore avec les principaux constructeurs automobiles mondiaux.

L'entreprise produit des composants « allégés » pour trois périmètres majeurs : *châssis*, *groupe motopropulseur* et *carrosserie/structures*. Pour réduire la masse à fonction équivalente, Mubea s'appuie notamment sur des aciers à haute résistance, des produits laminés à épaisseur variable

(*tailor rolled products*) et des composites renforcés de fibres (verre/carbone). Cette approche vise des gains mesurables en consommation/émissions et une amélioration des performances dynamiques, notamment pour les véhicules sportifs.

Mubea opère à l'échelle mondiale en **Europe**, dans les **Amériques** et en **Asie**, avec **58 sites** répartis dans **22 pays**, ce qui permet le développement et la production au plus près des usines des constructeurs.

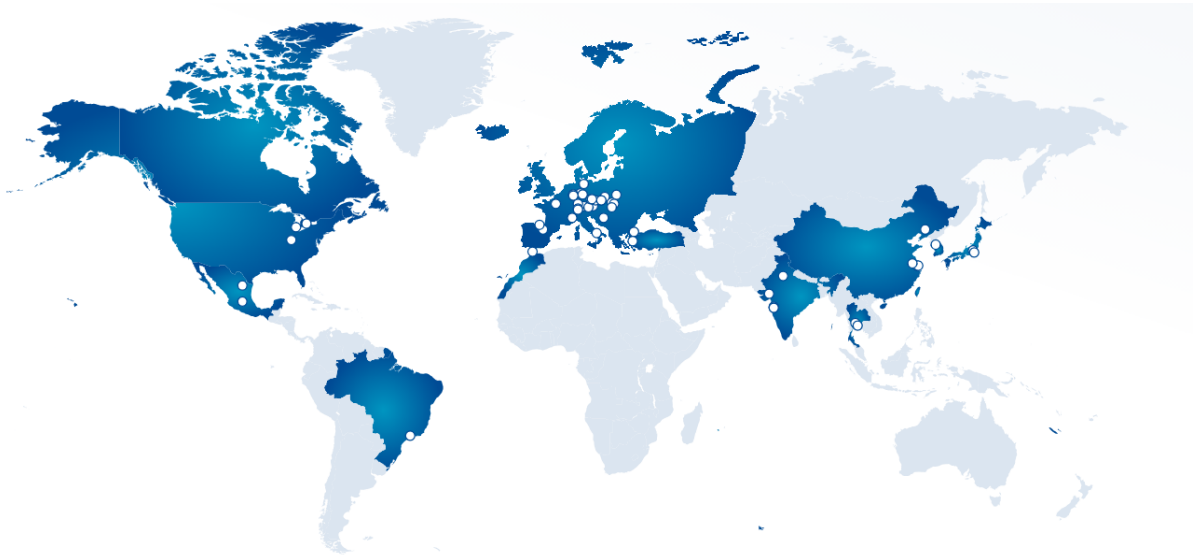

FIGURE 1 – Carte des sites de Mubea dans le monde. Source : [1]

2.2 Contexte du stage

Le stage s'est déroulé sur le site de Wendlingen am Neckar (Allemagne), au sein de la branche *Mubea Carbo Tech*, spécialisée dans l'assemblage de sièges en fibre de carbone pour véhicules sportifs. J'étais rattaché à l'équipe dirigée par Stefan Kompa, responsable de la gouvernance des processus et de la transformation numérique, en tant que stagiaire ingénieur.

La production est organisée en pipeline de stations interconnectées : les sièges se déplacent sur rails et sont assemblés étape par étape. Le site produit jusqu'à **environ 200 sièges par jour** et vise une augmentation de cette cadence.

La production est pilotée par des systèmes d'information industriels, dont *SAP*, qui gèrent les commandes, le suivi de fabrication, les instructions et la traçabilité. À ce jour, le monitoring des lignes s'effectue principalement via ces outils consultés par les opérateurs et l'encadrement.

3 Problématique et notions de base

Cette section précise la problématique industrielle à laquelle répond le projet, introduit les notions nécessaires (KPI, *Operational KPI Dashboard*, pipeline) et situe le rôle des couches MES/ERP/PLC qui alimentent la solution.

3.1 Problématique industrielle

Comme mentionné plus haut, les lignes d'assemblage de sièges carbone à Wendlingen fonctionnent en pipeline de stations. Le pilotage opérationnel repose aujourd'hui sur des systèmes existants (ERP/MES, dont *SAP*) et sur les écrans et rapports qui en sont issus. En pratique, les informations utiles au pilotage (cadence, arrêts, rebuts, retards, disponibilité) restent fragmentées, hétérogènes et parfois latentes, ce qui complique la détection précoce des écarts et la comparaison entre stations, équipes et sites.

Le besoin est de disposer d'un *suivi opérationnel unifié et temps réel* : des KPI définis de manière standard, calculés de façon homogène, visualisés clairement au poste comme au niveau atelier/site, et réplicables sur d'autres usines du groupe, sans remise en cause des systèmes en place.

Contraintes clés.

- **Temps quasi réel** pour les indicateurs critiques (arrêts, cadence, rebut).
- **Multi-site** : duplicable entre lignes et usines avec un paramétrage minimal.
- **Scalabilité** : maintien un haut débit de traitement malgré un volume important de messages.
- **Interopérabilité** : intégration avec tout ERP/MES en place.
- **Robustesse** : tolérance aux pannes partielles, reprise sans perte, disponibilité 24/7.
- **Évolutivité** : ajout/modification de KPI sans interruption ni refonte.

Le système visé ne remplace pas les fonctions cœur de l'ERP/MES (planification, exécution, qualité) ; il en agrège et valorise les données pour le monitoring opérationnel.

3.2 Définitions

Cette sous-section fixe le vocabulaire employé dans la suite du rapport : *KPI*, *Operational KPI Dashboard*, notion de *pipeline* de traitement, ainsi que les couches *MES/ERP/PLC*.

3.2.1 KPI

Un *KPI* (*Key Performance Indicator*) est un indicateur quantifiable qui relie l'activité opérationnelle à des objectifs métiers. Définis par une règle de calcul explicite, un périmètre (poste, ligne, période), une granularité et des règles de datation/agrégation, les KPI servent à **piloter** : suivi de la performance et des goulots, respect des délais, productivité/capacité, qualité (rebuts), et coûts associés. Ils doivent être comparables entre stations/équipes/sites, assortis de *cibles* et *seuils* (tolérances, alertes) pour permettre des décisions concrètes (réaffectation de ressources, maintenance, ajustement du takt (cadence cible dictée par la demande client), priorisation des ordres).

Pour le prototype, deux KPI **au niveau station** ont été retenus :

- le *temps de travail moyen de la station pour un siège* (cycle moyen au poste), indicateur direct du rythme et des goulots par rapport au takt visé.
- le *nombre total de sièges traités par la station* (débit cumulatif), reflet de la production réalisée versus le plan. Ce choix s'appuie sur les KPI effectivement déployés sur l'une des productions au cours du stage et répond à un double besoin métier : détecter rapidement les ralentissements locaux et quantifier l'atteinte des volumes.

3.2.2 Operational KPI Dashboard

Un *Operational KPI Dashboard* est un dispositif de suivi en **exploitation** qui présente, en quasi temps réel, des indicateurs clés standardisés (KPI) pour piloter l'activité au plus près du terrain (poste, ligne, atelier, site). Il vise l'**action immédiate** : détecter un écart, comprendre où et pourquoi il se produit, et engager la correction.

3.2.3 Pipeline

Une *pipeline* désigne un enchaînement ordonné d'étapes de traitement où la sortie de chacune devient l'entrée de la suivante. Elle sert à découpler les responsabilités, faciliter le parallélisme (pipelining) et rendre chaque étape remplaçable/observable via des interfaces stables, avec une communication possible en synchrone ou asynchrone.

Exemple simple : une pipeline de traitement de données — collecte de données brutes, nettoyage/normalisation, agrégation, chargement dans une base analytique, puis visualisation — qui transforme des logs en indicateurs exploitables.

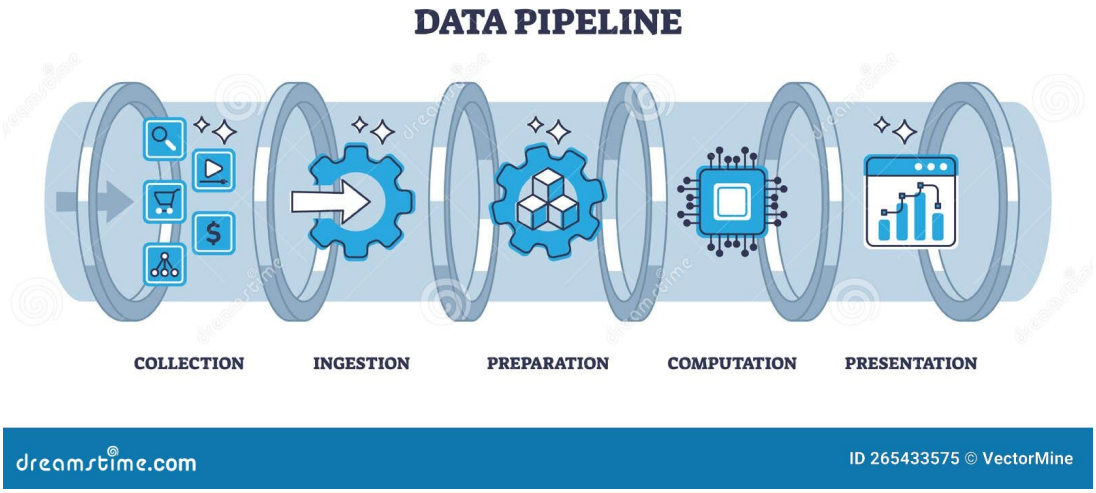

FIGURE 2 – Illustration d'une pipeline de traitement de données. Source : [2]

3.2.4 Couches MES / ERP / PLC

Dans un atelier de production, trois couches logicielles et matérielles se complètent. L'*ERP* (*Enterprise Resource Planning*) gère la planification et l'administration centrales : commandes, nomenclatures, gammes, capacités, ordres de fabrication, coûts. Le *MES* (*Manufacturing Execution System*) fait le lien atelier–ERP : il décline les ordres au poste, orchestre l'exécution (ordonnancement fin), collecte les états et assure la traçabilité. Le *PLC* (*Programmable Logic Controller*) pilote en temps réel les machines et capteurs/actionneurs au niveau station.

Ces couches s'articulent typiquement ainsi : l'ERP émet et suit les ordres, le MES les exécute et remonte les données d'atelier, tandis que les PLC fournissent les signaux temps réel nécessaires au contrôle et à la mesure. Les horizons temporels diffèrent — ERP (heure–jour), MES (minute–seconde), PLC (milliseconde) —, d'où la nécessité d'une agrégation cohérente.

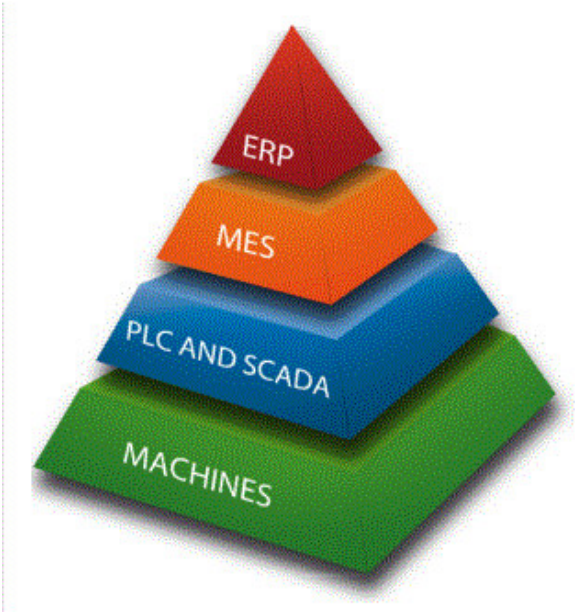

FIGURE 3 – Pyramide des couches ERP/MES/PLC. Source : [3]

Dans ce projet, le tableau de bord opérationnel n'a pas vocation à remplacer ERP/MES/-PLC : il exploite leurs flux (par exemple ceux du MES de type *SAP*) pour produire des KPI standardisés.

4 État de l'art

Cette section situe le projet parmi les principales familles d'outils de suivi industriel et précise leurs atouts/limites pour un besoin de KPI opérationnels quasi temps réel, **comparables entre stations et lignes d'un même site**.

Tableaux de bord ERP/MES. Intégrés nativement aux processus (ordres, gammes, traçabilité) et aux référentiels (articles, postes, utilisateurs), ils offrent une sécurité et une cohérence fortes.

- **Atouts** : données de référence fiables, droits/traçabilité intégrés, moins de « tuyauterie » à maintenir.
- **Limites** : personnalisation temps réel souvent coûteuse, rafraîchissement périodique (minutes), difficulté à standardiser et déployer **des tableaux homogènes site par site**, dépendance éditeur/licences.

Plateformes de BI (Business Intelligence). Excellentes pour le décisionnel périodique (quotidien/hebdomadaire) et l'analyse multi-sources.

- **Atouts** : modèles de données riches, visualisations avancées, partage/annotations, gouvernance du reporting.
- **Limites** : moins adaptées au suivi à la seconde (dataflows, caches), coût/latence d'actualisation, mise en œuvre peu naturelle pour des écrans d'atelier.

SCADA/HMI. Ciblent le pilotage local machine/ligne (temps réel milliseconde à seconde) avec une forte intégration automate/capteurs.

— **Atouts** : réactivité locale, alarmes, synoptiques proches du terrain.
— **Limites** : vision surtout *intra-ligne*; agrégation inter-lignes **au sein d'un site** faisable mais nécessitant une couche d'agrégation supplémentaire; standardisation des KPI plus difficile (hétérogénéité fournisseurs).

Architectures « composables » orientées flux. Empilent des briques spécialisées : bus/streams (transport d'événements), calcul de flux (micro-services), base de séries temporelles (stockage/requêtes), visualisation (dashboards).
— **Atouts** : modularité, faible latence, scalabilité horizontale, standardisation des schémas KPI, **réplicabilité site par site** sans imposer d'agrégation inter-sites.
— **Points d'attention** : gouvernance du schéma (définition/versionnage des KPI), fiabilité (reprise, idempotence), sécurité (authN/authZ des flux), observabilité (logs/metrics/traces) et exploitation (déploiement, sauvegardes).

Positionnement du présent travail. Le projet adopte cette approche composable : **Redis Streams** pour transporter des événements, modules C++ pour le quasi temps réel, **InfluxDB** pour les séries temporelles et **Grafana** pour la visualisation. L'objectif est un **suivi opérationnel par site** avec des KPI standardisés, faible latence et exploitation simple ; la comparaison ou consolidation inter-sites ne fait pas partie du périmètre actuel (elle pourrait être étudiée ultérieurement).

5 Conception et modélisation de la solution

Cette section présente les choix technologiques, l'architecture cible et les principes de **distributivité** et de **scalabilité**, avant de proposer des schémas d'ensemble.

5.1 Technologies utilisées

Les briques retenues et leur justification sont résumées ci-dessous.

5.1.1 Python

Python est un langage *interprété* et *haut niveau*, à typage dynamique, doté d'une bibliothèque standard riche et d'un vaste écosystème de paquets (PyPI). Multiplateforme, il est particulièrement adapté au *scripting*, aux conversions de données, aux outils en ligne de commande et au prototypage rapide, avec une syntaxe lisible et peu de plomberie (gestion de fichiers, JSON, HTTP, etc.).

Dans ce projet, Python a été retenu pour l'**outillage non critique en performance** : génération de données de test (p. ex. *json_generator.py*), scripts de benchmark, petits parseurs/conversions et utilitaires d'administration. Ce choix tient à la rapidité de développement, à la lisibilité du code et à la disponibilité de bibliothèques matures (clients Redis/HTTP, outillage CLI).

5.1.2 C++

C++ est un langage compilé, typé statiquement, conçu pour écrire du code « proche de la machine » tout en offrant des abstractions de haut niveau (STL, classes, génériques). Portables et performants, les programmes C++ permettent un contrôle fin de la mémoire, des structures de données et de la concurrence, avec un standard moderne (C++17) qui fournit les primitives nécessaires (threads, verrous, atomiques, conditions, fichiers, réseau).

Dans ce projet, C++ (C++17) a été retenu pour **tout le chemin temps réel** de la pipeline afin d'assurer une latence faible et un débit élevé, avec un comportement prévisible sous charge. Le langage permet d'optimiser les structures (p. ex. `std::string`, `std::unordered_map`) et de maîtriser l'allocation. Côté intégration, les modules utilisent un client C++ pour Redis Streams et publient vers InfluxDB au format *line* via HTTP. L'ensemble reste portable et déployable en services indépendants sur plusieurs machines.

5.1.3 Redis

Redis est un serveur clé-valeur en mémoire, léger et simple à déployer (un seul binaire) [4]. La fonctionnalité *Streams* fournit une **file d'événements ordonnée** (on ajoute à la fin, chaque entrée a un identifiant horodaté), pratique pour transporter des messages entre processus.

Dans ce projet, *Redis Streams* sert de bus de messages. Les producteurs publient avec `XADD` en envoyant directement une table clé–valeur (`std::unordered_map<std::string,std::string` — ex. `station_id:P05`, `timestamp:...`). Les consommateurs lisent avec `XREAD` en mode bloquant (avec *timeout*) en lui fournissant un ID : à chaque appel, `XREAD` renvoie les entrées **strictement postérieures** à l'ID fourni. Pour lire depuis le début d'un flux, on passe `"0"` (équivalent de « premier message ») ; ensuite, on réutilise l'ID du dernier message reçu. Côté C++, la structure renvoyée est du type `std::vector<std::pair<std::string, std::vector<std::pair<std::string, std::unordered_map<std::string,std::string>>>>>` : pour chaque flux (ou canal), une liste d'entrées {ID, map}. La `unordered_map` présente dans la structure est exactement celle envoyée via `XADD`, ce qui permet d'accéder immédiatement aux champs sans re-parsing.

Nous retenons Redis pour sa **simplicité de mise en œuvre** (un seul binaire), son **API courte** et ses **outils d'inspection** utiles au diagnostic (`XRANGE`, `XINFO`), qui accélèrent les essais et le dépannage tout en offrant une latence faible suffisante pour le besoin.

D'autres options existent mais sont moins adaptées à une prise en main rapide dans ce contexte : *Kafka* demande une infrastructure plus lourde à installer et opérer [5] ; *RabbitMQ* (AMQP) introduit des concepts supplémentaires (exchanges, bindings) utiles pour des routages complexes mais superflus ici [6].

5.1.4 InfluxDB

InfluxDB est une base de données de séries temporelles (TSDB) conçue pour ingérer rapidement des points horodatés. On écrit des **points** composés de : un *measurement* (nom logique), des **tags** (étiquettes indexées pour filtrer/agréger), des **fields** (valeurs métriques) et un *timestamp*. Les *tags* distinguent les **séries** : même measurement et mêmes tags ⇒ même série ; une valeur de tag différente ⇒ autre série, *toujours dans le même bucket (base)*. Les *fields* ne sont pas indexés et portent les valeurs. L'écriture/lecture se fait simplement en HTTP via le *line protocol* [7].

Dans ce projet, InfluxDB stocke les KPI calculés en temps quasi réel. Le *Kpi_pusher* écrit

via le *line protocol* (HTTP), avec une modélisation typique : `measurement` = `kpi`, `tags` = {`site`, `ligne`, `station`, `kpi_name`}, `fields` = {`value`}, `timestamp` fourni par le calcul. Cette structuration permet un filtrage efficace (par station/ligne/site/KPI) et une intégration directe dans Grafana pour la visualisation.

En pratique, trois points d'attention : (1) la **cardinalité des tags** — si un tag prend trop de valeurs différentes, l'index grossit et les requêtes ralentissent [8] ; on limite donc les tags aux dimensions stables (ligne/station/kpi_name) et on met le reste en *fields* ; (2) le **batching** des écritures — envoyer les points par paquets réduit le coût réseau/IO ; (3) le **downsampling** — résumer l'historique ancien (ex. moyennes/percentiles par minute puis par heure) pour contenir le volume. À l'échelle actuelle, ces points sont maîtrisés ; s'ils deviennent sensibles, des actions sont prévues dans les perspectives.

5.1.5 Grafana

Grafana est une plate-forme open source de tableaux de bord orientée séries temporelles [9]. Elle se connecte nativement à InfluxDB et permet de construire rapidement des vues avec variables (p. ex. `station`), filtres, auto-rafraîchissement et annotations. Les panneaux (graphiques, jauges, tableaux) s'assemblent sans développement front-end, et le mode « kiosk » facilite l'affichage sur écrans d'atelier. Des transformations côté lecture et un mécanisme d'alerting simple complètent l'usage au quotidien.

Ce choix privilégie une mise en place rapide, un suivi quasi temps réel et un accès à distance. Limites connues : pas de logique métier complexe dans les dashboards (elle reste dans les modules de calcul), peu de « write-back », et un rendu « pixel-perfect »/reporting limité.

5.2 Architecture globale

L'architecture adopte une approche orientée messages, où chaque module est un composant indépendant relié par un bus de données (Redis Streams). L'objectif est de découpler l'acquisition, la transformation, le routage, le calcul et la persistance des KPI, afin d'obtenir un fonctionnement quasi temps réel, une montée en charge par réplication de modules et une mise en production progressive par site/ligne.

Le *Connector* se connecte au système MES en place et extrait les événements de production, qu'il publie sous forme de chaîne. Le *Convertor* transforme ensuite chaque événement en table clé–valeur (`std::unordered_map`), format aligné avec le modèle *field–value* de Redis Streams pour faciliter le transfert et le traitement par les autres modules. Lorsque le nombre de stations est élevé, un *PreRouter* partitionne les flux par identifiant de station et les achemine vers un **pool de *Routers*** *dimensionné en fonction du nombre de stations* ; celles-ci sont **réparties de façon équilibrée** entre les *Routers* pour limiter la contention tout en **préservant l'ordre** au sein de chaque station. Chaque *Router* aiguille alors les messages de son sous-ensemble de stations vers le *Kpi_ engine* correspondant (et, si nécessaire, vers un flux global pour des KPI transverses). À l'inverse, lorsque le nombre de stations est limité, le *PreRouter* n'est pas nécessaire : un *Router* unique dessert l'ensemble des stations. Le *Kpi_ engine* applique les règles de calcul des indicateurs au niveau station. Enfin, le *Kpi_ pusher* formate les résultats et les écrit dans InfluxDB.

InfluxDB assure la persistance en séries temporelles (tags typiques : station, ligne, site), et Grafana restitue les indicateurs sous forme de tableaux de bord opérationnels (vues par station, agrégations par ligne, puis par site). Ce découpage modulaire permet de répliquer les modules sensibles au débit (par exemple *Router* ou *Kpi_ engine*), de partitionner les flux par station/ligne

et d'isoler les responsabilités pour la maintenance et l'évolution.

Concernant l'organisation Redis, l'architecture retient au minimum **deux serveurs** par ligne de production : (i) un Redis « amont » relie *Connector*→*Convertor*→(*PreRouter*)→*Router* ; (ii) un Redis « aval » relie *Router*→*Kpi_engine*→*Kpi_pusher*. Lorsque le nombre de stations est élevé et qu'un *PreRouter* est activé, l'amont reste unique, tandis que l'aval est **découpé** : **un serveur Redis par groupe de *Router*/*Kpi_engine*/*Kpi_pusher***. Cette séparation limite la contention, isole les pics de charge et préserve l'ordre à l'intérieur de chaque flux.

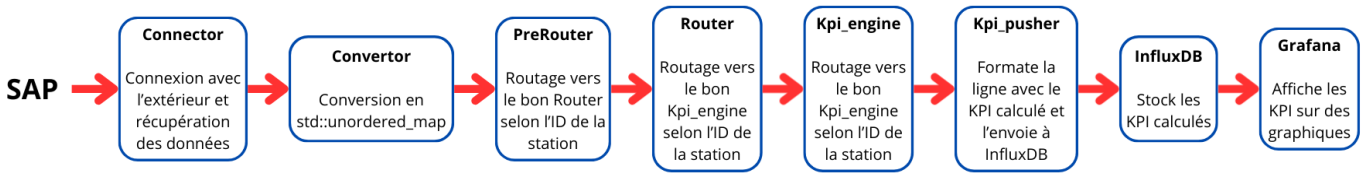

FIGURE 4 – Schéma simplifié de la pipeline

5.3 Distributivité et scalabilité

On distingue l'organisation **distribuée** du déploiement et la **scalabilité** ; la seconde s'appuie directement sur la première.

5.3.1 Distributivité

L'architecture est distribuée par conception : chaque instance de chaque module (Connector, Convertor, PreRouter, Router, Kpi_engine, Kpi_pusher) s'exécute comme un processus indépendant, potentiellement sur des machines différentes, et communique via un bus de messages (Redis Streams) distant. Les flux sont partitionnés par station afin de préserver l'ordre local là où il est nécessaire et d'isoler les défaillances. Ce découplage permet le déploiement progressif par site/ligne, la tolérance aux pannes partielles, ainsi que des mises à jour ou redéploiements sans interruption du reste de la chaîne.

5.3.2 Scalabilité

La scalabilité est obtenue principalement par *scale-out* sélectif des modules gourmands (p. ex. duplication de *Router* ou de *Kpi_engine*) et par le partitionnement des flux (sharding par identifiant de station). À l'échelle de l'usine, l'architecture prévoit **une pipeline complète par ligne de production** ; à l'échelle du groupe, **une base InfluxDB par site** isole les écritures et simplifie l'exploitation. Lorsque le nombre de stations augmente sur une ligne, le *PreRouter* répartit les messages vers plusieurs *Router* pour éviter la contention tout en garantissant l'ordre à l'intérieur d'une partition. Le *Kpi_pusher* peut paralléliser les écritures vers la base de séries temporelles, tandis que les consommateurs côté calcul restent idempotents pour absorber des reprises. Autrement dit, la **distributivité** n'est pas qu'un choix de déploiement : elle constitue le *levier* de la **scalabilité horizontale**, en permettant d'augmenter le débit et de maintenir une latence quasi temps réel malgré un grand nombre de messages.

5.4 Schéma d'architecture

Les schémas ci-dessous présentent une **version simplifiée** de la pipeline logique pour une seule ligne de production ; des schémas plus complets et détaillés sont fournis en annexe.

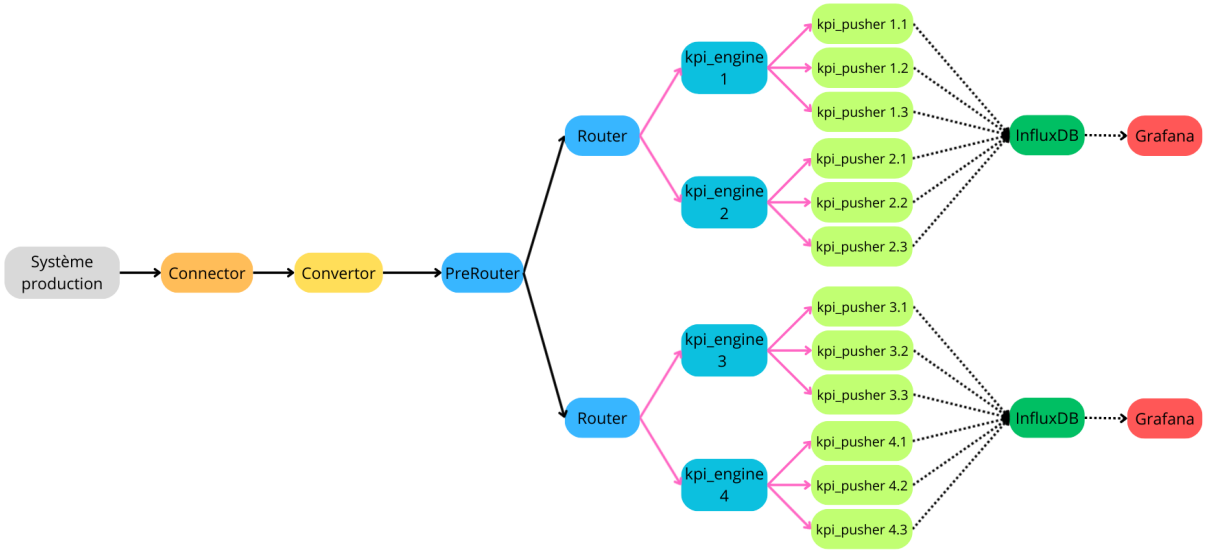

FIGURE 5 – Pipeline avec *PreRouter* pour une ligne à 4 stations et 3 KPI par station

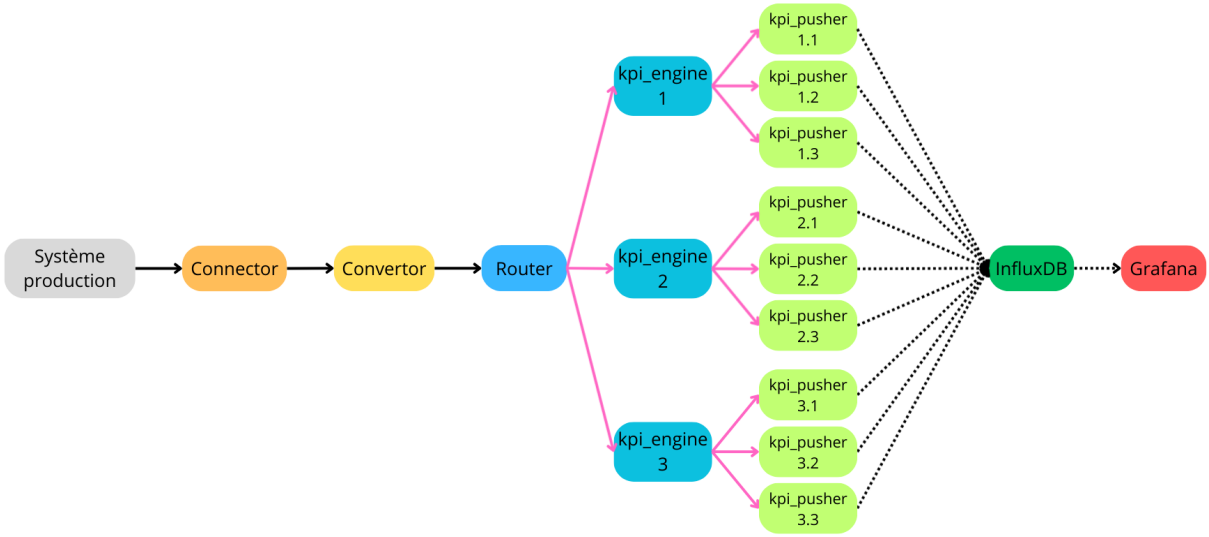

FIGURE 6 – Pipeline sans *PreRouter* pour une ligne à 3 stations et 3 KPI par station

6 Développement et modules

Cette section présente l'organisation du code, le fonctionnement global (*Fetcher* / *Worker*), le multithreading et la synchronisation, l'indépendance des modules, ainsi que leurs **main**, leurs fichiers de configuration et l'arrêt centralisé ; elle détaille ensuite chaque module — de l'ingestion à la visualisation — et leurs points d'extension.

6.1 Architecture et hiérarchie du code

Le projet est organisé comme suit (présentation synthétique des répertoires et fichiers) :

- `build/` — répertoire de sortie : artefacts de compilation et exécutables générés par CMake (build *out-of-source*).
- `config/` — fichiers JSON de configuration, classés par module (`config/Connector/`, `config/Router/`, ...).
- `extern/` — dépendances tierces vendorisées pour Redis (client/bibliothèques issus de GitHub), incluses pour faciliter la compilation sans installation système supplémentaire.
- `include/` — en-têtes de chaque module et `sharedcontrol.hpp` (présenté ci-après).
- `scripts/` — utilitaires Python, notamment `json_generator.py`.
- `src/` — code source :
  - un sous-dossier par module, chacun contenant `main.cpp` et `[nom_du_module].cpp`;
  - un `main.cpp` « global » au niveau de `src/`.
- `CMakeLists.txt` — script CMake principal.
- `README.md` — instructions d'usage rapides.

Compilation. Le projet utilise **CMake** pour orchestrer la compilation et la génération des exécutables [10] ; le fichier `CMakeLists.txt` à la racine décrit les cibles, les répertoires d'en-têtes et les liens avec les bibliothèques externes, et place les artefacts dans `build/`.

6.2 Fonctionnement global et multi-threading des modules

Tous les modules suivent le même patron : un objet *Fetcher* et un objet *Worker* (traitement). Le *Fetcher* exécute une boucle `run()` qui lit le canal Redis et, en cas d'absence de message (message vide/timeout), passe à l'itération suivante ; sinon, il sauvegarde l'identifiant du message et transmet au *Worker*. Le *Worker* exécute sa propre boucle `run()` : il prélève, traite, puis publie vers l'étape suivante. La synchronisation s'appuie sur les structures et protocoles définis dans le fichier `sharedcontrol.hpp`, présenté ci-après. Au démarrage, le constructeur du (*Fetcher*) instancie le *Worker* avec les bons paramètres et lance un thread dédié sur sa fonction `run()` qu'il détache (sauf pour *Connector*, où l'inverse s'applique. Plus de détails ci-après).

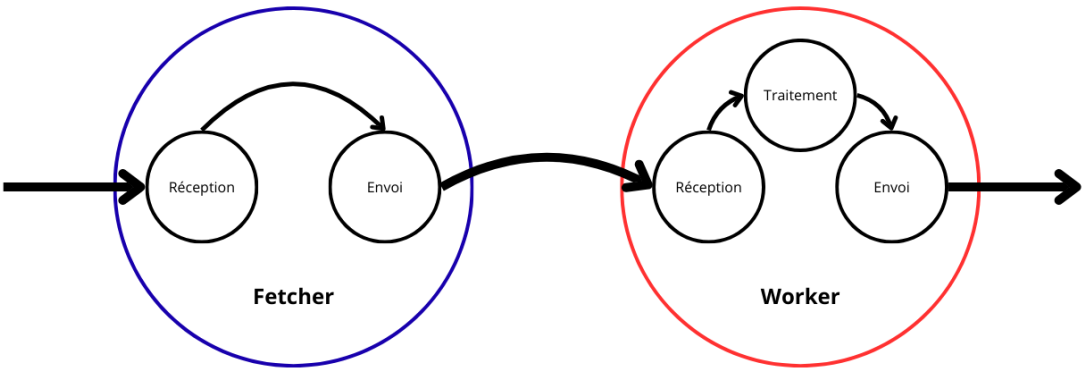

FIGURE 7 – Relation *Fetcher*/*Worker*

Lorsque plusieurs *traitements possibles* existent, le *Worker* suit un **patron de stratégie** : un objet de base factorise la fonction `run()` et les attributs communs, et un *sous-objet* spécialisé n'implémente que le `process` propre au traitement choisi. À l'initialisation du module, **un seul** sous-objet est instancié (en fonction du système/du format des données de la ligne de production en place) et c'est lui qui est lancé dans le thread *Worker*.

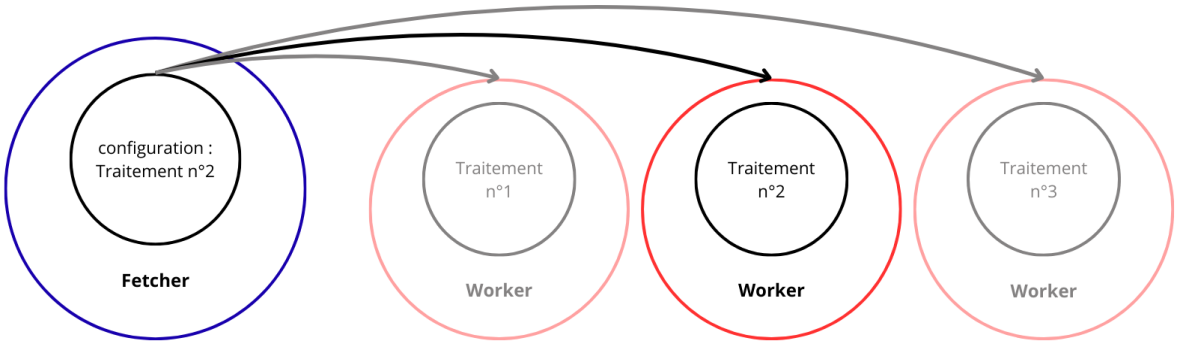

FIGURE 8 – **Un seul** *Worker* choisi selon la configuration du module

Lorsque plusieurs traitements **peuvent** être activés simultanément, le *Worker* conserve le même **patron de stratégie** : la classe de base factorise `run()` et les attributs communs, et chaque traitement est porté par un *sous-objet* spécialisé. À l'initialisation du module, on instancie alors **autant de threads *Worker*** que de traitements souhaités, le *Fetcher* assurant le dispatch.

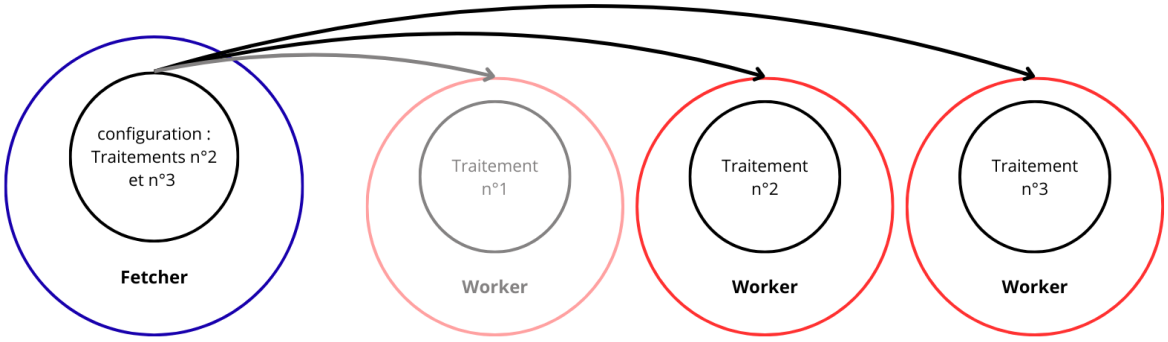

FIGURE 9 – **Plusieurs** *Workers* choisis selon la configuration du module

Concrètement, ajouter un traitement consiste à (i) implémenter un *sous-objet* spécialisé qui fournit le `process()` dédié, puis (ii) ajouter son instanciation/sélection dans le constructeur de l'objet principal du module (en fonction de la configuration).

Ce modèle permet d'ajouter/modifier un traitement sans impacter les autres, et contribue au maintien d'une faible latence malgré un débit élevé.

6.3 Synchronisation (`sharedcontrol.hpp`)

Ce fichier regroupe les structures communes utilisées par tous les modules pour la synchronisation **intra-module** (entre threads d'un même module).

Structures.

— `Child_` : contient `mutex`, `cond` (variable de condition) [11], `active` (booléen) et un `std::shared_ptr<const Message>` jouant le rôle de *buffer* Fetcher → Worker. Ici `Message` représente la structure lue depuis Redis.

— `Control_` : contient un `std::vector<std::shared_ptr<Child_>>` nommé `kids`, utilisé lorsque le *Fetcher* pilote plusieurs *Workers* en parallèle.

Deux modes d'utilisation. (1) Un seul *Worker*. À la construction, le *Fetcher* crée une instance `Child_` et en passe le pointeur au *Worker* (tout en le conservant). **(2) Plusieurs *Workers*.** Le *Fetcher* crée autant de `Child_` que de *Workers*, en passe un à chacun (passage de pointeurs), et stocke leurs pointeurs dans `Control_::kids`.

Protocole d'échange (côté *Fetcher*). Dans sa boucle `run()` et pour **chaque** *Worker* (cas 1 : unique ; cas 2 : boucle sur `kids`) :

1. verrouiller le `mutex` du *Worker* ;
2. attendre sur `cond` que `active == false` (le *Worker* a fini de lire la donnée précédente) ;
3. Affecter un *pointeur partagé* sur le nouveau `Message` au buffer du *Worker* ;
4. poser `active = true`, notifier le *Worker*, puis déverrouiller.

Protocole d'échange (côté *Worker*). Dans sa boucle `run()` :

1. verrouiller son `mutex` ;
2. attendre que `active == true` (le *Fetcher* a mit une nouvelle donnée à lire) ;
3. affecter le *pointeur partagé* du buffer à un *pointeur local* du thread ;
4. poser `active = false`, notifier le *Fetcher*, puis déverrouiller ;

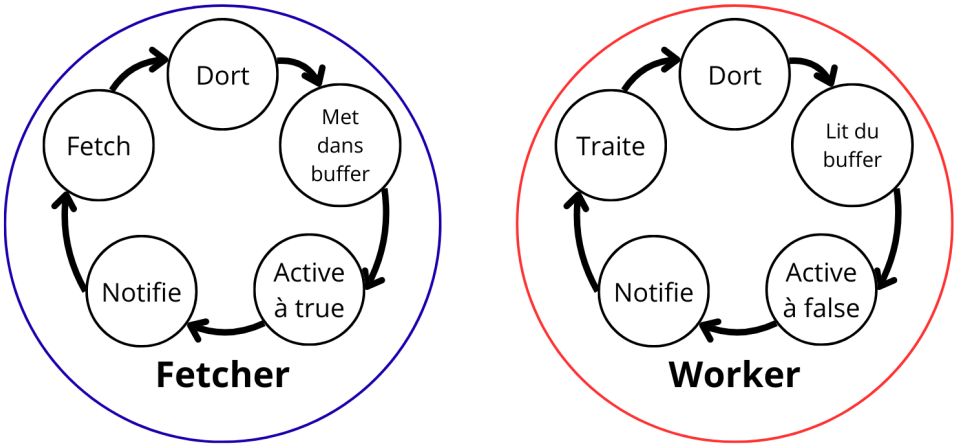

FIGURE 10 – Schéma du protocole de synchronisation *Fetcher*/*Worker*

Pourquoi un `Child_` par *Worker* ? Un *buffer*/verrou/condition **par *Worker*** permet de paralléliser les traitements sans interférence : le *Fetcher* peut alimenter à nouveau un *Worker* dès qu'il a copié localement, sans attendre les autres. Un verrou/file **unique** pour tous forcerait un séquencement global (goulot d'étranglement) et empêcherait d'avoir plusieurs *Workers* actifs simultanément.

Pourquoi des `shared_ptr<const Message>` ? Pour éviter les copies inutiles et simplifier la gestion de durée de vie. Le *Fetcher* crée un unique `Message` à chaque lecture Redis et transmet aux *Workers* un *pointeur partagé* vers cet objet. Chaque *Worker* lie ce pointeur à son pointeur local (traitement en lecture seule) ; le même bloc mémoire est donc exploité en parallèle. L'objet n'est libéré que lorsque le dernier pointeur local est relâché. Résultat : zéro copie de données, accès concurrents sûrs en lecture et gestion mémoire simple.

6.4 Indépendance des modules

Cette sous-section décrit comment chaque module est autonome au déploiement et à l'exécution : un binaire dédié piloté par un fichier de configuration JSON, et un mécanisme d'arrêt centralisé permettant un démarrage/arrêt propre indépendamment des autres modules.

6.4.1 Main et fichier de configuration

Chaque module est livré avec deux unités de compilation : (i) un fichier qui contient le code des objets du module (*Fetcher*/*Worker*, logique métier) ; (ii) un `main` minimal dédié au déploiement.

Le binaire du `module` reçoit en argument le *path* vers un fichier de configuration JSON (spécifique au module et à l'instance, en cas de réplication). Ce fichier, stocké dans `config/[NOM_DU_MODULE]/`, est chargé puis converti en `std::unordered_map` pour une manipulation directe des paramètres. Le `main` instancie l'objet principal du module avec ces paramètres, puis lance sa boucle `run()` sur un thread dédié qu'il détache.

Cette organisation rend chaque module indépendant et facilement distribuable : déployer ou répliquer un module revient à fournir un fichier JSON d'instance et à lancer un nouveau processus pointant vers ce fichier.

6.4.2 Arrêt centralisé

Chaque `main` de chaque module déclare un *pointeur partagé* vers un booléen `stop_requested`, passé à l'objet du module. Un *signal handler* pour `SIGINT` (Ctrl+C) positionne ce booléen à `true`. Après l'instanciation, le `main` entre dans une boucle d'attente légère (sleep court) tant que `stop_requested` est `false`, évitant toute surcharge CPU.

Côté objets (*Fetcher*/*Worker*), l'arrêt se traite ainsi :

- **Fetcher** : (i) *au début de l'itération*, avant un éventuel `continue` (message vide/timeout), tester `stop_requested` ; si vrai, notifier le *worker*/les *workers* puis quitter ;
  (ii) *dans le prédicat d'attente* (`cond.wait(...)` avec `!active || stop_requested`) : si l'arrêt est demandé, ne pas attendre — ou en sortir aussitôt —, notifier le *worker*/les *workers*, puis quitter.
- **Worker** : *dans le prédicat d'attente* (avec `active || stop_requested`) ; si l'arrêt est demandé, ne pas attendre — ou en sortir aussitôt —, notifier le *fetcher*, puis quitter.

L'opération de lecture bloquante `XREAD` utilise un *timeout* borné afin de réévaluer périodiquement le drapeau d'arrêt. Ce mécanisme évite les interblocages de fin de vie et garantit un *shutdown* propre et déterministe de chaque instance.

6.5 Connector

Rôle. *Connector* fait l'interface entre la pipeline et le système de production en place (MES/ERP ou équivalent — p. ex. SAP, LASE) : il **récupère** les événements émis par la ligne et les **pousse** vers la suite de la chaîne.

Conception (inversion *Fetcher*/*Worker*). Contrairement aux autres modules où la logique de lecture est fixe (Redis) et où seul le traitement varie, ici c'est **la lecture qui dépend du système** (SAP, LASE, etc.), tandis que le traitement est trivial (publier tel quel en `string`). Par conséquent, le *Worker* est l'« objet principal » (initiateur) : il **sélectionne le backend** de lecture, **instancie** le *Fetcher* adapté, lui passe sa structure `Child_` et **lance** son thread `run()`. Le *Worker* exécute ensuite sa propre `run()` pour relayer les messages.

Objets et responsabilités.

- `Connector` (**Worker**) : boucle `run()` qui prélève les messages reçus du *Fetcher* via `Child_`, puis `processMessage()` qui **publie en** `string` vers Redis. Le message est sérialisé en `std::string` et emballé dans une map `std::unordered_map<std::string,std::string>` avec un unique champ `data: payload` .
- `BaseConnector` (**Fetcher de base**) : factorise la boucle `run()` de lecture (attentes, notifications) et déclare les points d'extension `connect()` et `fetchMessage()`.
- `[sap]`/`[lase]` (**Fetchers spécialisés**) : implémentent `connect()` (établissement de la session) et `fetchMessage()` (récupération d'un événement). *Remarque* : au stade prototype, aucune logique de connexion n'est implémentée, les *Fetchers* lisent depuis un flux Redis d'entrée pour simuler le MES.

Configuration et E/S. Le module reçoit les paramètres suivants (via fichier JSON) :

- `redis_input_uri` : URI du Redis d'entrée (mode test) ;
- `input_stream` : nom du flux d'où lire (mode test) ;
- `redis_output_uri` : URI du Redis entre *Connector* et *Convertor* ;
- `system` : identifiant du système en place (`sap`, `lase`, ...) pour choisir le *Fetcher* ;
- `stop_requested` : drapeau d'arrêt partagé.

Le **flux de sortie** est construit à partir de `input_stream` en suffixant `CV_` (pour *ConVertor*), par exemple `WEN_POR_CN_CV_`.

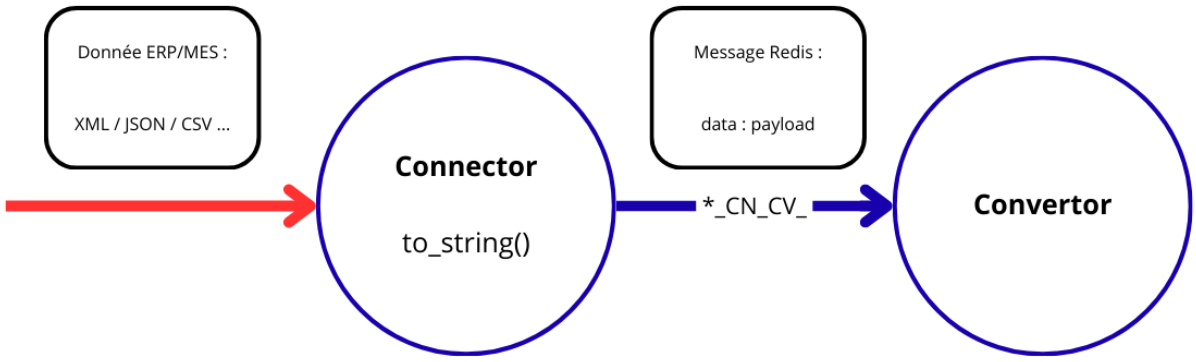

FIGURE 11 – Schéma du comportement du Connector

6.6 Convertor

Rôle. *Convertor* reçoit les messages bruts publiés par *Connector* (payload texte) et les transforme en une représentation **clé–valeur** exploitable par la suite de la pipeline. L'objectif est d'exposer directement, sous forme de paires `{clé:valeur}`, les champs utiles (p. ex. `station_id`, `ts`, ...).

Conception (Fetcher principal + stratégie de parsing). Ici, la **lecture** est fixe (depuis Redis) alors que le **parsing** peut varier selon le format d'entrée ; le *Fetcher* est donc l'initiateur. L'objet principal `Convertor` exécute `run()` en mode *Fetcher* (lecture Redis), **instancie** un *Worker* en fonction du `format` demandé et lui passe sa structure `Child_`. Le *Worker* dérive d'une classe de base `BaseParser` (qui factorise la boucle `run()`) ; un sous-objet spécialisé fournit le `processMessage()` adapté (au stade de prototype, seul le format JSON est pris en charge).

Objets et responsabilités.

- `Convertor` (**Fetcher**) : `run()` lit sur Redis (conserve *last_id*), achemine le message vers le *Worker*.
- `BaseParser` (**Worker de base**) : factorise `run()` et déclare `processMessage()`.
- `JsonParser` (**Worker spécialisé**) : implémente `processMessage()` en parsant le *string* JSON et en l'aplatissant via `flattenJson()` vers une `std::unordered_map` avant de le publier vers le PreRouter (ou le Router directement).

Configuration et E/S. Paramètres (via JSON) :

- `redis_input_uri` : URI du Redis entre Connector et Convertor ;
- `input_stream` : nom du flux d'où lire ;
- `redis_output_uri` : URI du Redis entre Convertor et PreRouter (ou Router) ;
- `format` : identification du format des données (`json`, `xml`, `csv`, ...) pour choisir le *Worker*;
- `stop_requested` : drapeau d'arrêt partagé.

Le **flux de sortie** est dérivé de `input_stream` en ajoutant le suffixe `_PR` (pour *PreRouter*), par ex. `WEN_POR_CN_CV_PR_`.

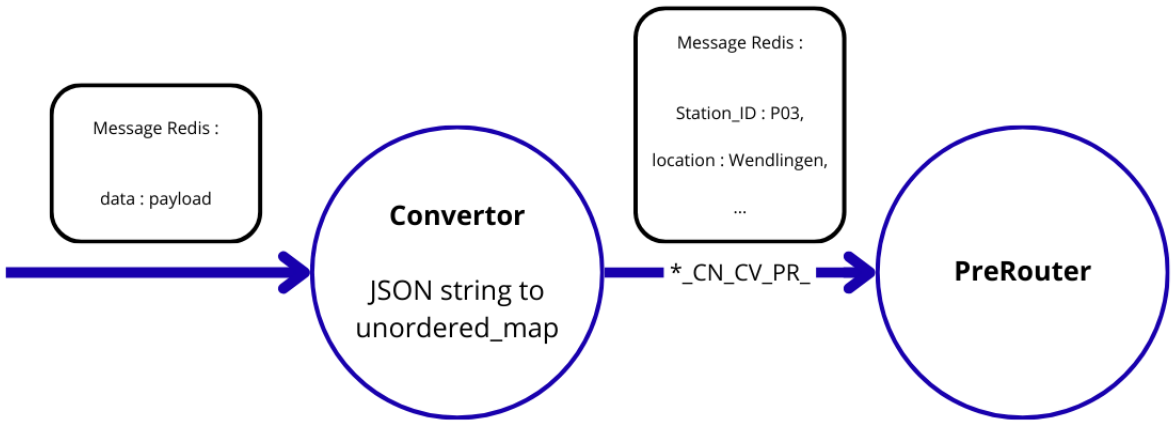

FIGURE 12 – Schéma du comportement du Convertor

6.7 PreRouter

Rôle et conditions d'activation. *PreRouter* n'intervient que lorsque le **nombre de stations est élevé**. Il **partitionne** le flux en amont des *Router* pour répartir la charge sur un *pool* de *Router*, en préservant l'ordre *par station* et en limitant la contention.

Conception (Fetcher principal + Worker de routage). L'objet principal `PreRouter` joue le rôle de *Fetcher* : il lit depuis Redis et alimente un *Worker* `ChildRouter` chargé de **diriger chaque message** vers le bon canal en fonction d'une *routing key* (champ du message Redis, typiquement `station_id`).

Objets et responsabilités.

- `PreRouter` (**Fetcher**) : boucle `run()`, lecture sur Redis puis remise des messages au `ChildRouter`.
- `ChildRouter` (**Worker**) : `run()` + `processMessage()` ; extrait la valeur de `routing_key` et choisit le **canal de sortie** via une table `redis_outputs` (`std::unordered_map`) qui associe chaque valeur à un *affixe* de flux. Le canal est composé en **suffixant** l'`input_stream` avec cet affixe (p. ex. `"R1_"`), puis publication par `XADD`.

Configuration et E/S.

- `redis_input_uri`, `input_stream` : lecture du flux amont (`..._PR`) ;
- `redis_output_uri` : serveur Redis côté *Router* ;
- `redis_outputs` : table {valeur de `routing_key` → affixe de flux, p. ex. `"P07"` → `"R1_"`} ;
- `routing_key` : nom du champ utilisé pour router (ex. `station_id`) ;
- `stop_requested` : drapeau d'arrêt partagé.

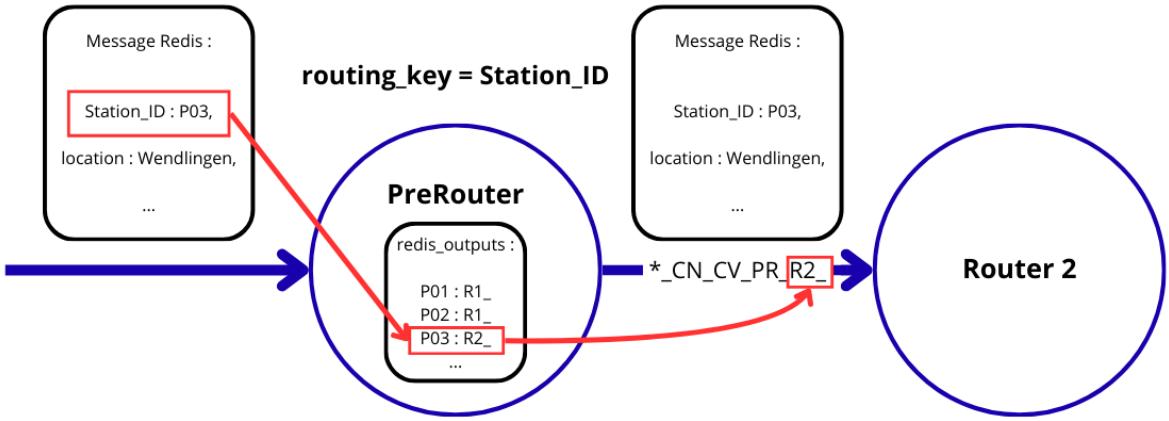

FIGURE 13 – Schéma du comportement du PreRouter

6.8 Router

Rôle. *Router* distribue les messages vers le(s) *Kpi_ engine*(s). Il route selon une *routing key* (p. ex. `station_id`) en **suffixant** directement la valeur du champ au `input_stream` pour former le canal de sortie station (préservation de l'ordre d'arrivée pour une station donnée). Il peut également dupliquer chaque message vers un *Kpi_ engine* **global** pour des KPI transverses.

Modes de fonctionnement. Deux modes activables indépendamment : **local** et **global**. Les deux peuvent être actifs simultanément : le même message est publié sur le flux local et sur le flux global.

Conception (Fetcher principal, workers local/global). L'objet principal `Router` opère en *Fetcher* : il lit depuis Redis puis alimente un ou deux *Workers* selon la configuration :

- un **worker local** qui calcule le canal `out_stream = input_stream + valeur(routing_key)` et publie vers le Redis « local » (vers *Kpi_ engine* de la station) ;
- un **worker global** qui publie tel quel vers un `global_output_stream` dédié (sans routage par clé).

Objets et responsabilités.

- `Router` (**Fetcher**) : `run()` multi-enfants (attente/dispatch/notifications) et `fetchMessage()` depuis le flux amont.
- `BaseRouter` (**Worker de base**) : factorise `run()` (prise des messages via `Child_`, publication), déclare `processMessage()`.
- `LocalRouter` (**Worker local**) : `processMessage()` lit la `routing_key`, compose `out_stream` pour chaque message en **suffixant** la valeur au `input_stream`, puis publie via `XADD` sur `local_redis_output_uri`.
- `GlobalRouter` (**Worker global**) : `processMessage()` publie sans routage vers `global_output_stream` (Redis de sortie global).

Configuration et E/S. Paramètres (JSON) :

- `redis_input_uri` : Redis entre *PreRouter*/*Convertor* et *Router* ;
- `input_stream` : flux amont à lire ;
- `local_redis_output_uri` : Redis entre *Router* et *Kpi_ engine* (mode local) ;
- `routing_key` : champ de routage (ex. `station_id`) ;
- `global_redis_output_uri` : Redis pour le *Kpi_ engine* global ;
- `global_output_stream` : flux global de sortie ;
- `local` / `global` (booléens) : activation des modes ;
- `stop_requested` : drapeau d'arrêt partagé.

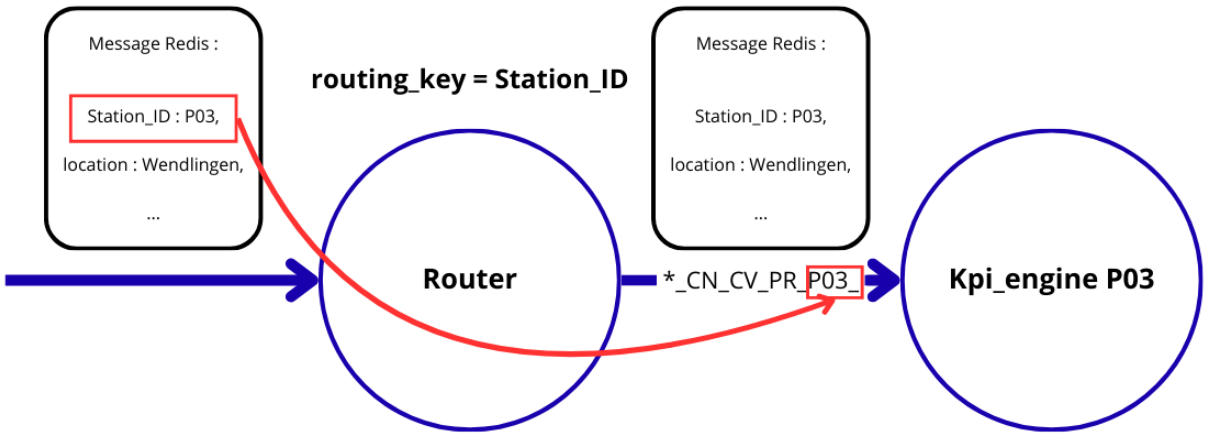

FIGURE 14 – Schéma du comportement du Router

6.9 Kpi_engine

Rôle. *Kpi_engine* calcule les indicateurs au niveau **station** en quasi temps réel. Pour chaque station, il applique les règles métiers (compteurs, temps moyens, ...) et publie le résultat vers un *Kpi_pusher* dédié.

Conception (Fetcher multi-enfants, un worker par KPI). L'objet principal `KpiEngine` opère en *Fetcher/Dispatcher* : il lit le flux de la station (via `XREAD`, *timeout*, *last_id*) puis **alimente en parallèle** autant de *Workers* que de KPI demandés. Chaque *Worker* calcule son indicateur de façon sérialisée (ordre respecté pour la station) et publie immédiatement le point vers la sortie. Ainsi, on parallélise **entre** KPI tout en conservant l'ordre **dans** chaque KPI.

Objets et responsabilités.

- `KpiEngine` (**Fetcher/Dispatcher**) : `run()` multi-enfants (attente, dispatch, notifications) et `fetchMessage()` depuis le flux de la station.
- `BaseKpi` (**Worker de base**) : factorise `run()` et introduit une **gestion quotidienne** des KPI (voir ci-dessous), déclare les points d'extension `processMessage()` et `reset()`.
- `TotalSeatCount` / `AverageWorkTime` (**Workers spécialisés**) : implémentent `processMessage()` (calcule de l'indicateur) et `reset()` (remise à zéro quotidienne), puis publient vers leur flux de sortie.

Remise à zéro quotidienne (minuit). Les KPI sont calculés **par journée**. `BaseKpi::run()` maintient un `next_midnight` (timestamp du prochain minuit). À chaque itération, le worker vérifie l'horloge : si l'instant courant dépasse `next_midnight`, il appelle `reset()` (spécifique au KPI) puis recalcule `next_midnight` (minuit suivant). Ce mécanisme est **centralisé** dans `BaseKpi` pour éviter de le dupliquer dans chaque KPI.

Configuration et E/S. Paramètres (JSON) :

- `redis_input_uri` : Redis entre *Router* et *Kpi_engine* (flux de la station) ;
- `input_stream` : préfixe du flux station (issu du *Router*) ;
- `redis_output_uri` : Redis entre *Kpi_engine* et *Kpi_pusher* ;
- `kpis` : liste des KPI à activer (p. ex. `["total_seat_count","average_work_time"]`) ;
- `stop_requested` : drapeau d'arrêt partagé.

Le **flux de sortie** d'un KPI est construit comme `output_stream = input_stream + "_" + kpi_name` (un flux par {station,KPI}).

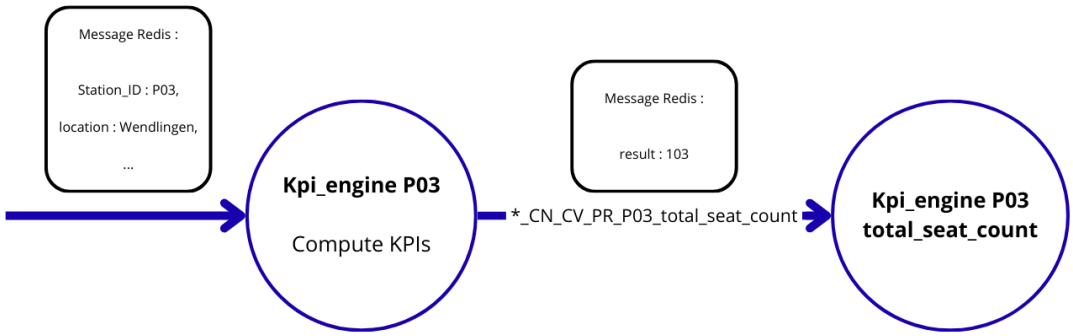

FIGURE 15 – Schéma du comportement du Kpi_engine

6.10 Kpi_pusher

Rôle. *Kpi_pusher* assure la **persistance** des KPI calculés : il consomme les résultats produits par *Kpi_engine* et les écrit dans InfluxDB au format *line protocol*.

Conception (Fetcher mono-enfant). Le module suit un schéma *Fetcher/Worker* minimal : un **Fetcher** unique lit le flux Redis et alimente un seul **Worker** (`ChildPusher`) chargé de formater la ligne et de l'envoyer à InfluxDB. Ce découplage garde l'E/S bloquante hors du fil de publication.

Objets et responsabilités.

- `KpiPusher` (**Fetcher**) : `run()` lit via `XREAD` puis remet chaque message au `ChildPusher`.
- `ChildPusher` (**Worker**) : `run()` + `processMessage()` ; compose une ligne *line protocol* avec la **valeur reçue** et un **timestamp** en nanosecondes (fonction `now_ns()`), puis effectue l'envoi HTTP vers InfluxDB via `sendToInflux()`.

Configuration et E/S. Paramètres (JSON) :

- `redis_input_uri` : Redis entre *Kpi_engine* et *Kpi_pusher* ;
- `input_stream` : flux d'entrée (un par couple {station, KPI}) ;
- `location`, `production`, `station_id`, `kpi` : métadonnées utilisées pour composer la ligne *line protocol* ;
- `stop_requested` : drapeau d'arrêt partagé.

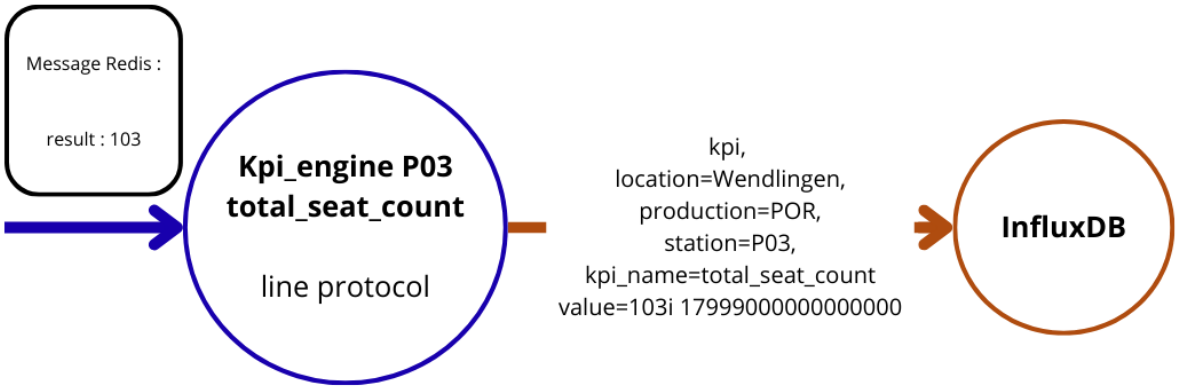

FIGURE 16 – Schéma du comportement du Kpi_pusher

6.11 Stockage InfluxDB

Rôle. Cette brique persiste les KPI calculés par *Kpi_engine* sous forme de points horodatés, directement exploitables par Grafana.

Modélisation et format d'écriture. Chaque point est écrit dans le *measurement* `kpi` avec :

- **tags** : `location`, `line`, `station`, `kpi_name` (dimensions stables pour filtrer/agréger) ;
- **field** : `value` (valeur du KPI) ;
- **timestamp** : horodatage en nanosecondes.

L'écriture se fait via le *line protocol* (HTTP). Le format produit par *Kpi_pusher* est équivalent à :

```
kpi,location=loc ,line=ligne ,station=id ,kpi_name=kpi  value=v i ts_ns
```

où `v i` indique une **valeur entière** et `ts_ns` est fourni par `now_ns()`.

Déploiement (prototype). InfluxDB est **déployée localement** via Docker (environnement de test) [12], avec un unique bucket `opkpi_raw`. L'accès se fait en local (port 8086 **non exposé** vers l'extérieur) — suffisant au stade prototype et cohérent avec l'absence de multi-site. Grafana se connecte ensuite à ce bucket pour les tableaux de bord.

6.12 Front-end Grafana

Objectif. Offrir une visualisation claire, hiérarchique et temps réel des KPI sans exposer la complexité du back-end. L'interface sert autant à l'exploitation quotidienne qu'au diagnostic (vue usine → ligne → station → détail).

Structure des dashboards.

- **A — Usines / Locations** : mosaïque de tuiles par site ; un clic ouvre les lignes de la localisation.
- **B — Lignes** : tuiles par ligne de production ; navigation en un clic vers les stations.
- **C — Stations** : tuiles par station ; chaque tuile ouvre la vue détaillée de la station.
- **D — Station KPIs** : deux panneaux par station (un *Stat* pour la dernière valeur, un *Time series* pour l'évolution).

Comportement et UX.

- **Navigation hiérarchique** par tuiles cliquables (A→B→C→D).
- **Auto-découverte** : nouveaux sites/lignes/stations détectés automatiquement dès qu'ils apparaissent dans les données.
- **Actualisation** : les panneaux chiffrés et les courbes se rafraîchissent à intervalle régulier ; les listes (usines/lignes/stations) se rechargent périodiquement.
- **Lisibilité** : tuiles homogènes avec titres centrés (Ligne BMW, Station P07) et pages station focalisées sur deux KPI clés.

Modèles de déploiement (latence vs couverture).

- **Grafana local sur site** : latence minimale (Grafana ↔ InfluxDB locaux) ; idéal pour l'équipe de production locale.
- **Grafana local hors site, relié à un InfluxDB distant** : latence réseau mais vision *multi-sites* dans une même interface (centre de supervision).
- **Grafana accessible par Internet** : monitoring depuis partout.
  Pratique : pour un seul site à distance, se connecter au *Grafana local* du site (réactif) ; pour plusieurs sites, se connecter à un *Grafana central* relié à plusieurs InfluxDB distants (plus de latence, mais vision consolidée).

7 Validation et benchmarks

7.1 `main.cpp` (lanceur du prototype)

Rôle. Le `main.cpp` sert d'**orchestrateur local** : il assiste l'utilisateur pour décrire une ou plusieurs **lignes** (canal MES de test, nombre de stations, liste de KPI), génère les fichiers de configuration JSON de chaque module, puis **démarre** une pipeline par ligne en créant un *processus* par module.

Flux d'utilisation (interaction terminal).

1. Demande du **nom du canal Redis** source (ex. `WEN_POR_sap`).
2. Demande du **nombre de stations** (entier).
3. Sélection des **KPI** parmi la liste proposée (ex. `total_seat_count`, `average_work_time`) avec possibilité d'en ajouter plusieurs.
4. Proposition d'**ajouter une autre ligne** et boucle sur les étapes 1–3 si oui.

Topologie et génération des configurations. Pour chaque ligne déclarée, le lanceur :

— **déduit la topologie** : seuil `STATIONS_PER_ROUTER` $\Rightarrow$ nombre de *Router* $= \lceil \#\text{stations} / \text{seuil} \rceil$; si > 1, activation d'un *PreRouter* et **répartition équilibrée** des stations entre *Router* ;

— **compose les noms de flux** selon la convention (par ex. suffixes `CV_` pour *Convertor*, `PR_` pour *PreRouter* (ou pour *Router* si pas de *PreRouter*), `R1_`, `R2_`... pour les *Router*, etc.) ;

— **génère un JSON** par instance de module sous `config/<module>/<instance>.json` (URIs Redis amont/aval, `input_stream`, `routing_key`, indicateurs activés, flags `local/global`, etc.).

Lancement des modules (fork/exec). Le lanceur itère sur les configurations et, pour chaque instance :

1. **fork** le processus ;
2. dans le *child*, **execl** le binaire du module en lui passant `argv[1]` = chemin du JSON ;
3. dans le *parent*, **mémorise le PID** de l'enfant.

Chaque ligne obtient ainsi sa pipeline complète (processus indépendants), avec séparation Redis *amont/aval* telle que décrite précédemment.

Arrêt propre et collecte des processus. Le lanceur installe un **handler SIGINT** (Ctrl+C) qui arme un drapeau d'arrêt. À la réception du signal :

1. il **relaye SIGINT à tous les PIDs** enfants (pour déclencher le `stop_requested` dans chaque module) ;
2. il **attend la terminaison** de chacun via `waitpid()` afin d'éviter les *zombies* (avec, si besoin, une politique d'escalade temporisée vers `SIGTERM`/`SIGKILL`).

Ce mécanisme s'aligne sur l'**arrêt centralisé** des modules : chaque module se réveille, notifie ses threads en attente, ferme ses boucles `run()` et quitte proprement [13].

7.2 `json_generator.py`

Rôle et usage. `json_generator.py` simule un flux d'événements **MES** afin d'alimenter la pipeline en l'absence de source réelle. Le script pousse des messages **JSON** dans des *Redis Streams* dédiés, ce qui permet de tester *Connector* (mode prototype), *Convertor* et le reste du chemin jusqu'à InfluxDB/Grafana.

Format des événements. Chaque message (champ `event` sérialisé en JSON) suit le schéma minimal :

```
{
  source,
  production,
  location,
  event_type,
  station_id,
  timestamp,
  data {
    order_number,
    part_serial_number,
    status
  }
}
```

Les `event_type` utilisés sont : `start_seat`, `work_in_progress`, `station_completed`. Les messages sont envoyés respectivement sur `WEN_POR_sap_CN_` (source `sap`) et `WEN_BMW_lase_CN_` (source `lase`).

Logique de génération. Le script initialise un **horodatage** à 08:00:00 du jour courant, puis alterne des incréments aléatoires pour simuler la cadence :

- **SAP** : pour **deux stations** tirées aléatoirement (`P01`–`P18`), il génère un `start_seat`, ensuite une séquence de 8 `work_in_progress` (+5 à +15 s chacun), puis `station_completed` (+5 à +15 s).
- **LASE** : pour **une station** (`P01`–`P11`), même protocole : `start_seat` → 8 `work_in_progress` → `station_completed`.

Chaque `start_seat` décale l'horloge de +20 à +40 s (aléatoire), puis chaque étape suivante ajoute un pas de +5 à +15 s. Les identifiants (`station_id`, `order_number`, `part_serial_number`) sont régénérés à chaque séquence. Une boucle finale répète ce pattern (par défaut 1000 itérations) pour produire un flux continu.

Envoi dans Redis. Les événements sont publiés avec `XADD` sous la forme `{ "event": <JSON> }`. Cette enveloppe unique est volontaire : elle simplifie le prototypage (*Connector* lit un champ `event` string) tout en restant compatible avec la suite (*Convertor* transforme ensuite en paires clé–valeur).

Limites et pistes. Les scénarios couvrent le « nominal » (pas de pannes, `status=ok`). Pour des tests plus riches : variabilité par station/ligne, événements d'arrêt/reprise, erreurs de format, et paramétrage externe (nombre de stations, cadence, durée).

7.3 Tests et mesures de performances

Cette section présente la méthodologie de test mise en place pour valider le bon fonctionnement du prototype, ainsi que les mesures de performances réalisées afin d'évaluer sa capacité à traiter un flux continu de messages en temps quasi réel.

7.3.1 Environnement de test

Les tests et benchmarks ont été réalisés sur la configuration suivante :

— **Matériel** : processeur *AMD Ryzen 5 5600H, 6 cœurs @ 3.30 GHz*, mémoire *16 Go RAM*, disque *SSD 475Go*.
— **Système d'exploitation** : *Windows 11 Famille, version 23H2*.
— **Redis** : version *7.0.15* déployée en local via *installation native*, configuration *par défaut*.
— **InfluxDB** : version *2.7*, déployée en *Docker*.
— **Grafana** : version *10.4.2*, mode de déploiement *Docker*.
— **Compilation du projet** : CMake *3.28.3* avec g++ *13.3.0*, standard C++17 activé.
— **Scripts auxiliaires** : Python *3.12.3*, bibliothèque `redis-py` pour le générateur de données (`json_generator.py`).

7.3.2 Tests et validation fonctionnelle

Les principaux tests sont résumés ci-dessous. Chaque ligne décrit le scénario, la méthode de test, puis le résultat attendu selon cette méthode.

Test	**Méthode**	**Attendu**	**Résultat**
Connexion du *Connector* à Redis	Démarrer *Connector* avec un stream de test et publier via `json_generator.py`. Vérifier avec `XRANGE/XINFO`.	Les événements apparaissent sur le stream configuré, sans erreurs de publication.	OK
Conversion JSON → `unordered_map` (*Convertor*)	Publier un JSON type ; tracer côté *Convertor* et lire en aval (via *Router*).	Le message converti est une table clé–valeur exploitable, champs attendus présents.	OK
Routage par station (*Router* local)	Envoyer des messages avec différents `station_id`; observer les streams destinés aux *Kpi_ engine*.	Chaque message va uniquement vers le *Kpi_ engine* de la station ciblée.	OK
Routage global (*Router* global)	Activer le mode global et publier un flux mixte. Contrôler le stream global.	Tous les messages sont aussi copiés vers le stream global.	OK
Calcul KPI `total_seat_count`	Générer des séquences `start_seat` → ... → `station_completed`.	Le compteur s'incrémente d'1 à chaque `station_completed` par station.	OK
Calcul KPI `average_work_time`	Générer des cycles avec temps contrôlés ; comparer à une moyenne calculée à la main.	La moyenne publiée correspond (à l'arrondi près) au calcul de référence.	OK
Reset quotidien des KPI (*Kpi_ engine*)	Simuler un passage de minuit (horodatage) puis publier de nouveaux messages.	Les KPI mémoire sont remis à zéro une fois au franchissement du seuil quotidien.	OK
Écriture InfluxDB (*Kpi_ pusher*)	Lancer la chaîne complète, puis interroger le bucket `opkpi_raw`.	Les points existent et sont observables dans le bucket avec les bonnes valeurs}.	OK
Arrêt propre (SIGINT)	Lancer plusieurs modules, envoyer `SIGINT` au lanceur.	Le lanceur affiche sur le terminage le bon déroulement de waitpid sur tout les processus.	OK

Table 1 – Validation fonctionnelle — méthode et résultats attendus

7.3.3 Benchmarks (débit end-to-end)

Périmètre et charge. Mesures réalisées **sur une seule machine** avec Redis/InfluxDB locaux (cf. environnement de test). Pipeline : une ligne de production, **un seul Router** (sans PreRouter), **2 KPI** calculés par station (`total_seat_count`, `average_work_time`). Pour $N \in \{10, 25, 50, 100, 200\}$ stations, on traite un volume proportionnel (10 000, 25 000, 50 000, 100 000, 200 000 messages). Le **débit end-to-end** est compté à l'**écriture dans InfluxDB** (point effectivement persisté).

Résultats (Router seul). Le débit **reste stable** à environ **2 300 msg/s** de bout en bout, **malgré l'augmentation du nombre de stations**.

# stations	# messages	Débit mesuré (msg/s)
10	10 000	≈ 2 300
25	25 000	≈ 2 300
50	50 000	≈ 2 300
100	100 000	≈ 2 300
200	200 000	≈ 2 300

Table 2 – Débit end-to-end avec **un Router**, 2 KPI par station, InfluxDB locale.

Avec PreRouter (partition logique). En activant un *PreRouter* (cas de **50 stations**, **10 stations/Router**, soit 5 Routers) sur la même machine et avec Redis local, le débit observé descend à ≈1 800 **msg/s** pour 50 000 messages. (cas de **30 stations**, **3 stations/Router**, soit 10 Routers) sur la même machine et avec Redis local, le débit observé descend à ≈1 650 **msg/s** pour 30 000 messages. Ces résultats **ne remetent pas en cause l'intérêt du PreRouter** : son objectif principal est la **répartition sur plusieurs serveurs Redis** afin d'éviter la contention *en environnement distribué*. Sur une machine unique, l'overhead supplémentaire (routage + multiples producteurs/consommateurs sur le même hôte) peut dominer et réduire le débit mesuré.

Lecture des résultats.

- **Stabilité du débit** avec un Router : la charge se répartit bien au niveau des KPI par station ; les écritures InfluxDB locales ne deviennent pas le goulot sur ces volumes.
- **Effet PreRouter en local** : la baisse mesurée reflète surtout le coût additionnel sur un seul hôte.
- **Perspective** : répéter ces tests avec Redis distribués (un par segment) pour évaluer le **gain réel en conditions visées** (réduction de contention, parallélisme inter-nœuds).

Limites. Tests effectués en local (réseau et disques non saturés), sans injection d'erreurs ni contention volontaire sur InfluxDB. Les **latences détaillées** (p50/p95/p99) n'ont pas été incluses ici ; elles pourront être ajoutées via des compteurs dans *Kpi_pusher* (horodatage `now_ns`) et un tableau de bord dédié.

8 Conclusion

La problématique traitée dans ce stage était claire : *fournir un suivi opérationnel unifié et en quasi temps réel des lignes de production, au travers de KPI standardisés, lisibles au poste comme à l'atelier, et suffisamment flexibles pour accueillir de nouveaux indicateurs — sans remplacer l'ERP/MES mais en valorisant leurs flux.* Autrement dit, transformer des données hétérogènes et parfois latentes en signaux exploitables immédiatement pour le pilotage.

Pour y répondre, nous avons conçu et implémenté une pipeline **distribuée et scalable** : des modules indépendants (acquisition *Connector*, conversion *Convertor*, *PreRouter*/*Router*, calcul *Kpi_engine*, persistance *Kpi_pusher*) communiquant via Redis Streams, stockant en InfluxDB et visualisant dans Grafana. Chaque module est **configurable par JSON**, exécutable en service, et doté d'un **multithreading** adapté (découplage lecture/traitement, synchronisation simple, ordonnancement par station). Le *Kpi_engine* gère la **remise à zéro quotidienne** et l'extension par *sous-objets* (patron de stratégie) pour ajouter de nouveaux KPI sans impacter le reste. Deux KPI station représentatifs ont été implémentés de bout en bout — `total_seat_count` et `average_work_time` — avec un schéma de tags (station/ligne/site) permettant des vues poste, ligne et site.

Côté performances, sur une ligne avec **un Router** et **2 KPI par station**, le débit **end-to-end** (jusqu'à l'écriture InfluxDB) se stabilise à **∼2 300 msg/s**, et reste **constant** lorsque le nombre de stations croît de 10 à 200 (volume proportionnel). En local, l'ajout d'un *PreRouter* introduit un surcoût (∼1 800 msg/s mesurés dans notre cas), ce qui est cohérent avec son rôle premier en environnement distribué : **répartir la charge** sur plusieurs serveurs et soutenir la montée en échelle inter-nœuds.

En synthèse, le projet livre un **socle opérationnel** qui satisfait la problématique posée : une architecture **modulaire, évolutive, distribuée et scalable**, déployable par ligne puis par site, et prête à accueillir de nouveaux KPI sans refonte.

9 Perspectives et améliorations possibles

Cette section esquisse les axes prioritaires pour passer du prototype à une solution exploitable et confortable à déployer.

9.1 Orchestrateur

Packaging en services/applications. Une alternative au lancement par `fork/exec` serait de packager chaque module comme un **service indépendant** (via `systemd` ou Docker). Le cycle de vie (démarrage, arrêt, relance) serait alors géré par le système, supprimant la gestion manuelle des `pid` et signaux. Cette approche simplifie l'orchestration, standardise l'arrêt propre (`stop_requested` via `SIGTERM`) et facilite le déploiement multi-sites, au prix d'un besoin de configuration centralisée (fichiers JSON/manifests).

Exécution distante. Objectif : pouvoir **déployer et lancer les modules sur d'autres machines** (même site), depuis un poste de pilotage.

— **Copie distante** des exécutables et fichiers de configuration (`scp`/`rsync`), puis **lancement via** `ssh`.

— Démarrage **sans zombification** : exécuter en service détaché (`systemd -user`, `nohup+&`, ou script « agent » distant) et journaliser (fichiers ou `journald`).
— **Compatibilité binaire** à vérifier (même OS/libc/arch) ; sinon, empaquetage léger (à étudier).
— Topologie réseau : URIs Redis *atteignables* entre hôtes, ports ouverts et authentification minimale.

Arrêt distant. Aujourd'hui, l'arrêt est local (`SIGINT`). Proposer un **plan de contrôle** Redis dédié *par site*.
— Un **Redis « control »** séparé ; **un canal par module** (p. ex. `ctrl:<module_id>`) sous forme de *stream*.
— Chaque `main` **bloque en** `XREAD` sur son canal de contrôle ; à réception d'un message, il positionne `stop_requested=true` et sort proprement (notification des threads, drain).
— Le **lanceur général** conserve la table {`module_id` → `ctrl_stream`} et **publie un message de stop** pour chaque module au lieu d'envoyer `SIGINT`.

UI. Concevoir une **interface de déploiement** (CLI/WEB) pour :
— décrire une ligne (canal, #stations, KPI), **générer les JSON** et déployer sur hôtes distants ;
— piloter **start/stop/restart** module par module ;
— afficher un **état synthétique** (dernier heartbeat, logs récents, latence/débit de base).
Objectif : rendre les opérations **plus sûres et moins manuelles**, avec une porte ouverte vers d'autres actions (redéploiement, changement de config).

9.2 Optimisations techniques

Optimisation bas-niveau du code. Le code actuel est fonctionnel mais loin d'être optimisé au maximum. De nombreuses améliorations bas niveau sont probablement possibles (réduction des copies, gestion plus fine de la mémoire, simplification de certaines boucles, etc.). Un travail futur pourrait consister à analyser et affiner ces aspects pour améliorer encore les performances globales.

Réplication des modules non-réplicables. Certains modules, comme *Connector* ou *Convertor*, ne sont pas conçus pour être répliqués en parallèle car ils doivent préserver l'ordre des messages sans routage intermédiaire. Une amélioration future consisterait à réfléchir à des mécanismes permettant de les paralléliser tout en maintenant cette garantie, sans devoir analyser le contenu des données de manière coûteuse.

Redis batching en lecture / pipeline en ajout. Augmenter **XREAD count** pour lire en lot, réduire le coût par message et amortir les réveils; côté écriture, regrouper plusieurs `XADD` via *pipeline* pour diminuer les allers-retours réseau. Ajuster `BLOCK` (timeout borné) pour concilier latence et débit.

InfluxDB batching / sharding. Écrire en **batch** (plusieurs points par requête HTTP) pour amortir le coût réseau, et activer une *retention policy* avec *downsampling* pour l'historique. Chaque site dispose déjà de sa propre base InfluxDB ; si le volume croît trop dans un site donné,

un **sharding intra-site** (plusieurs buckets par site, répartis par ligne ou par type de KPI) permettra de contenir la cardinalité et de maintenir les performances.

Format canonique pour la data. Définir un **schéma commun** en sortie de *Convertor* (noms de champs stables : `station_id`, `ts`, `event_type`, ...), quelle que soit la source (SAP, LASE, etc.). Fournir des *mappers* par source → schéma canonique. Bénéfice : modules aval (routage, KPI) **inchangés** quel que soit l'amont, et règles KPI portables entre lignes/sites.

9.3 Sécurité et robustesse

Accès restreint aux données/Dashboards. Mettre en place une gestion des droits d'accès afin que chaque utilisateur ne voie que les tableaux de bord ou données qui le concernent. Par exemple, un manager de site aurait accès uniquement aux dashboards de son usine, tandis qu'un directeur pourrait consulter ceux de tous les sites.

Tolérance aux pannes et réplication Redis. Introduire des mécanismes de réplication et de bascule automatique pour Redis afin d'assurer la continuité du service même en cas de panne d'un serveur.

Gestion des erreurs et résilience des modules. Améliorer la gestion des erreurs pour éviter qu'un module entier s'arrête à cause d'une anomalie ponctuelle. Chaque module devrait pouvoir détecter, consigner et, si possible, surmonter les erreurs.

Logging avancé et centralisé. Mettre en place et centraliser des journaux d'exécution des modules pour faciliter le suivi, le diagnostic et la maintenance, avec un historique consultable.

Gestion des potentielles fuites mémoire. Analyser le code à l'aide d'outils comme `valgrind` pour identifier et corriger les fuites mémoire éventuelles, afin de garantir la stabilité du système dans la durée.

Ajout d'un mode de debug. Prévoir un mode de débogage activable qui fournirait plus de détails dans les journaux, afin de faciliter les tests et l'investigation lors d'un problème.

9.4 Automatisation

Un axe d'amélioration serait d'automatiser davantage certaines opérations de déploiement et de gestion. Par exemple, permettre à *PreRouter* et *Router* de déterminer automatiquement le nombre de stations par *Router*, voire de décider si un seul *Router* suffit. D'autres pistes incluent la mise en place de mécanismes de redémarrage automatique en cas de panne (*reboot*), ou encore la possibilité de brancher/débrancher des modules à chaud (*hotplug*) sans interrompre l'ensemble du système.

9.5 Documentation technique

README.md. Un fichier `README.md` détaillé reste à rédiger afin de présenter clairement l'architecture, les dépendances et les instructions d'utilisation.

Commentary complet. Un commentaire complet du code source doit être ajouté pour faciliter la compréhension, la maintenance et l'évolution du projet.

Annexes

extraits de code

Listing 1 – Fetcher run() monoworker

```
void run() {
    while (1) {
        if (stop_requested_->load()) {
            child_->cv.notify_one();
            return;
        }
        Message messages;
        fetchMessage(messages);
        if (messages.empty()) {
            std::this_thread::sleep_for(std::chrono::milliseconds(100));
                // Avoid busy waiting
            continue;
        }
        last_id_ = messages.back().second.back().first;
        auto messages_ = std::make_shared<const Message>(std::move(
            messages));
        {
            std::unique_lock<std::mutex> lk(child_->m);
            child_->cv.wait(lk, [this]() {
                return !child_->active || stop_requested_->load();
            });
            if (stop_requested_->load()) {
                child_->cv.notify_one();
                return;
            }
            child_->messages = messages_;
            child_->active = true;
            child_->cv.notify_one();
        }
    }
}
```

Listing 2 – Fetcher run() multiworker

```
void run() {
    while (1) {
        if (stop_requested_->load()) {
            for (auto& child_ : control_->kids) child_->cv.notify_one();
            return;
        }
        Message messages;
```

```
        fetchMessage(messages);
        if (messages.empty()) {
            std::this_thread::sleep_for(std::chrono::milliseconds(100));
                // Avoid busy waiting
            continue;
        }
        last_id_ = messages.back().second.back().first;
        auto messages_ = std::make_shared<const Message>(std::move(
            messages));
        for (auto& child_ : control_->kids) {
            std::unique_lock<std::mutex> lk(child_->m);
            child_->cv.wait(lk, [this, child = child_]() {
                return !child->active || stop_requested_->load();
            });
            if (stop_requested_->load()) {
                for (auto& child_ : control_->kids) child_->cv.
                    notify_one();
                return;
            }
            child_->messages = messages_;
            child_->active = true;
            child_->cv.notify_one();
        }
    }
}
```

Listing 3 – Worker run()

```
void run() {
    while (1) {
        {
            std::unique_lock<std::mutex> lk(child_->m);
            child_->cv.wait(lk, [this]() {
                return child_->active || stop_requested_->load();
            });
            if (stop_requested_->load()) {
                child_->cv.notify_one();
                return;
            }
            messages_ = child_->messages;
            child_->active = false;
            child_->cv.notify_one();
        }
        processMessage();
    }
}
```

Schémas détaillés

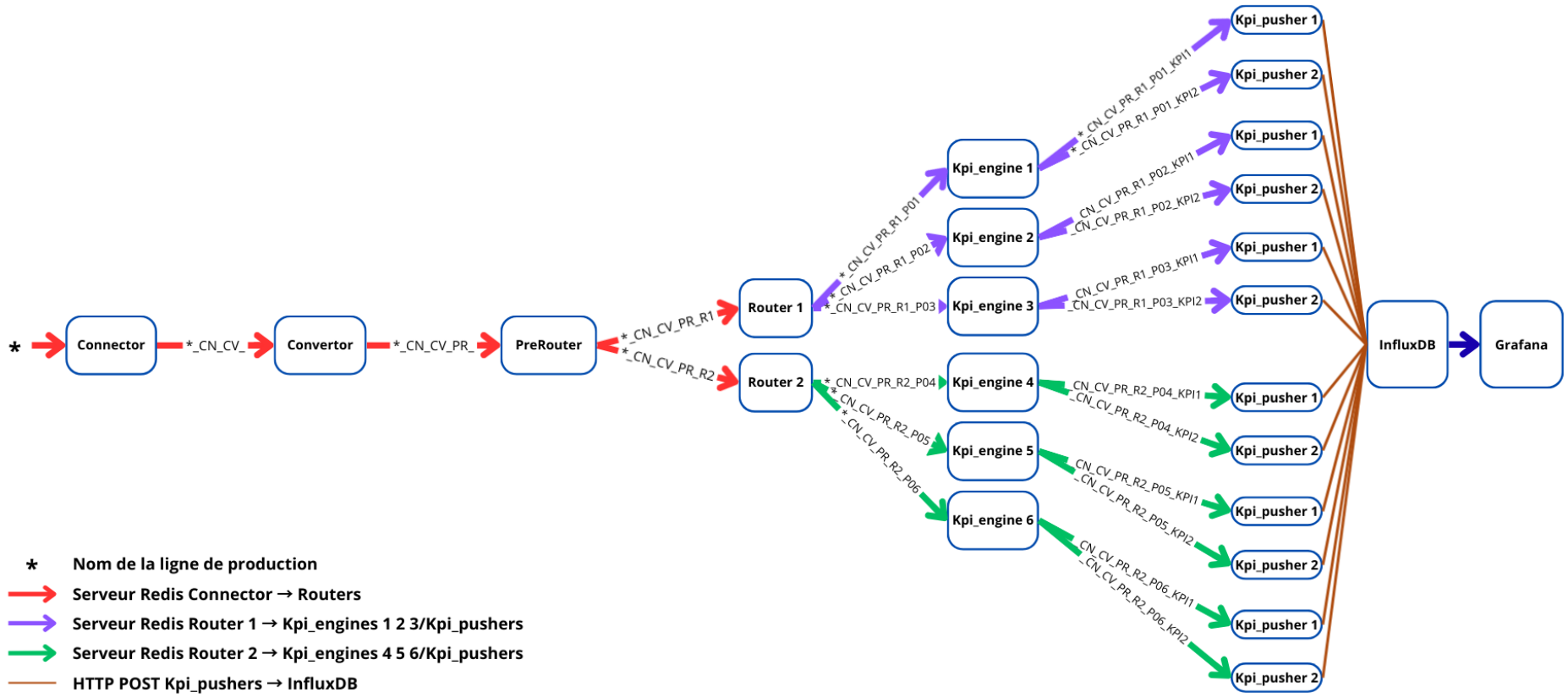

FIGURE 17 – Schéma détaillé d'une pipeline avec PreRouter pour une ligne à 6 stations et 2 KPI par station

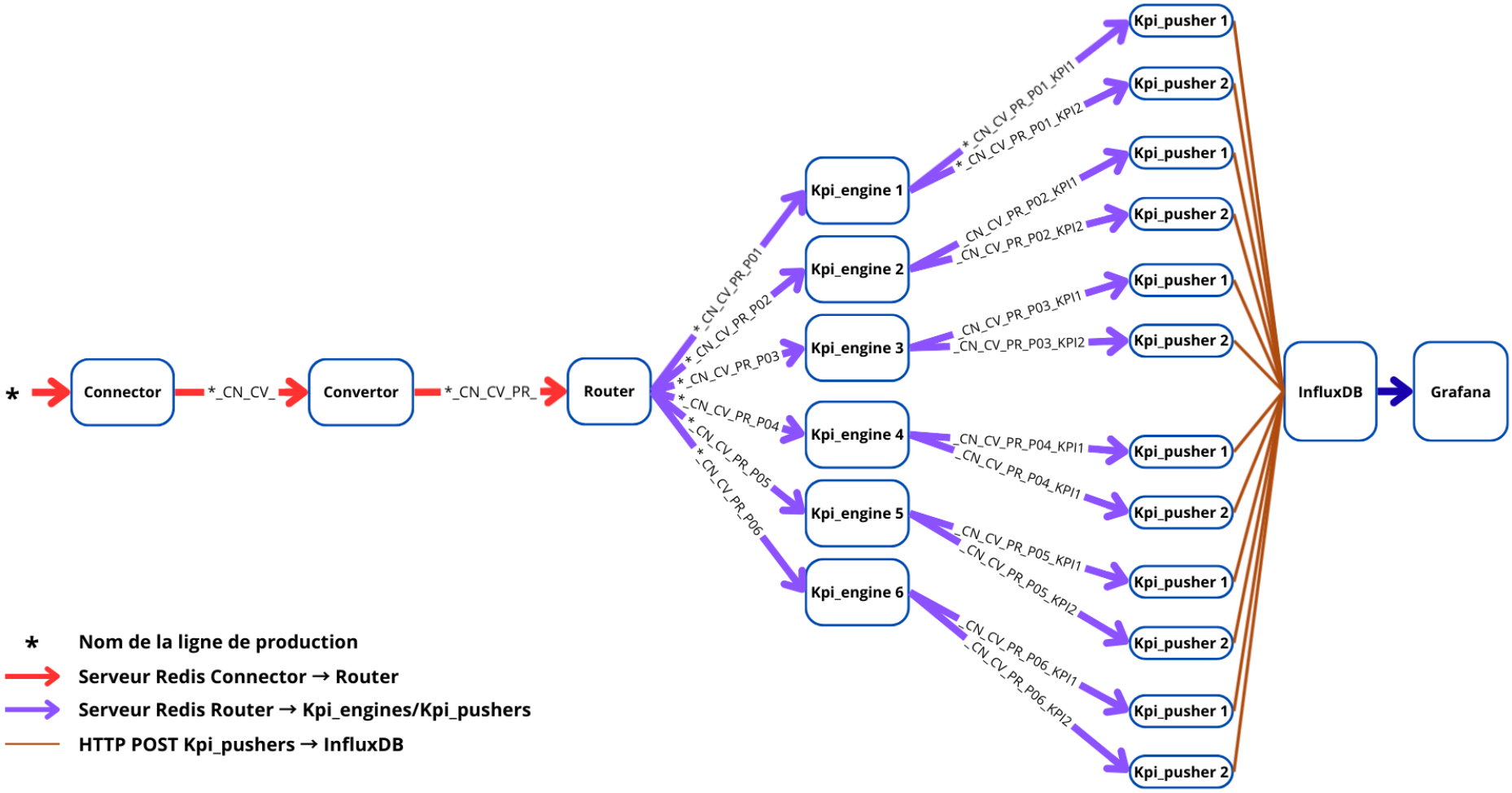

FIGURE 18 – Schéma détaillé d'une pipeline sans PreRouter pour une ligne à 6 stations et 2 KPI par station

Screenshots Grafana

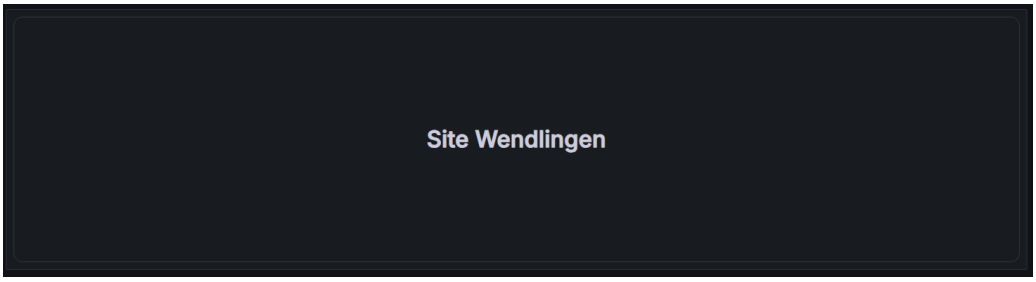

FIGURE 19 – Screenshot d'un Dashboard de type A

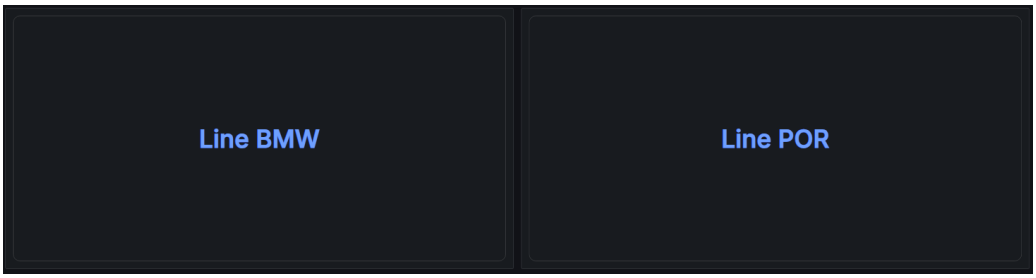

FIGURE 20 – Screenshot d'un Dashboard de type B

FIGURE 21 – Screenshot d'un Dashboard de type C

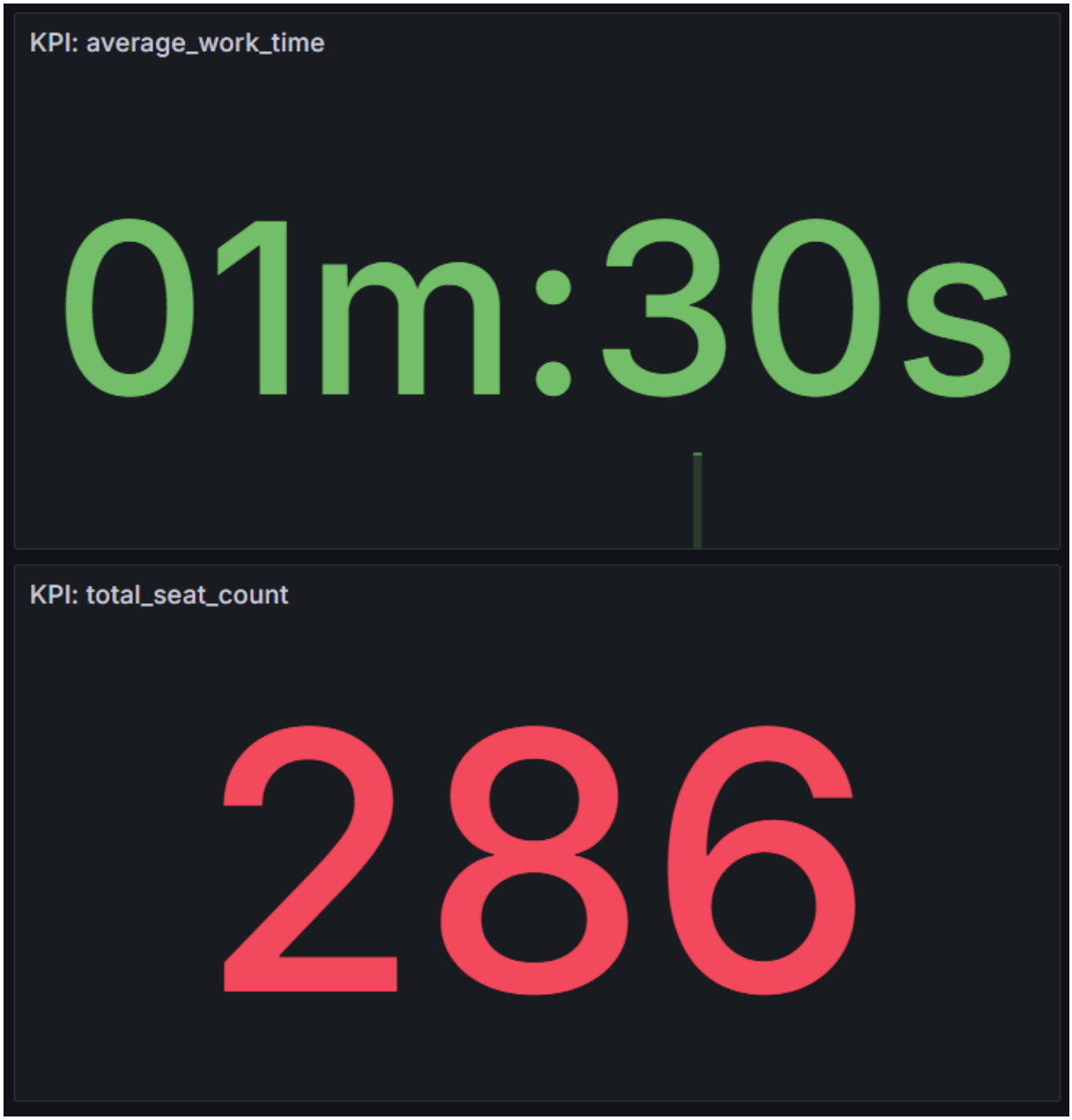

FIGURE 22 – Screenshot d'un Dashboard de type D

Références

[1] "Represented worldwide." `https://www.mubea.com/en/represented-worldwide`, 2025. Capture d'écran du site Mubea.

[2] I. 265433575, "Pipeline de données avec calcul des étapes du processus de préparation de fichier plan schéma.." dreamstime. Image téléchargée ; licence : libre de droits vectorielles).

[3] C. Cantin, "Pourquoi et comment réussir son projet mes ?." LinkedIn, 2016. Téléchargée d'un article LinkedIn.

[4] "Redis streams documentation." Documentation officielle (concepts et commandes Streams).

[5] "Apache kafka documentation." Documentation officielle (topics, partitions, consommateurs).

[6] "Rabbitmq documentation." Documentation officielle (AMQP, exchanges, queues, bindings).

[7] "Influxdb line protocol." Documentation officielle (format d'écriture HTTP).

[8] "Series cardinality in influxdb." Documentation officielle (impact des tags et bonnes pratiques).

[9] "Grafana documentation." Documentation officielle (datasources, panels, variables, kiosk).

[10] "Cmake documentation." Configuration et génération multi-plateforme.

[11] "std::condition_variable." cppreference. Synchronisation par mutex/variable de condition en C++.

[12] "Docker documentation." Conteneurisation et exécution locale d'InfluxDB/Grafana.

[13] "Signals (signal)." Gestion des signaux POSIX, usage de SIGINT pour l'arrêt propre.